TeamBook candiositos club de fans

Índice

1. Math **1**
- 1.1. Catalan . . . 1
- 1.2. extended . . . 1
- 1.3. fastgcd . . . 1
- 1.4. fft . . . 1
- 1.5. gauss_algebra_lineal . . . 2
- 1.6. linearsieve . . . 3
- 1.7. matrizFibo . . . 3
- 1.8. multiply . . . 3
- 1.9. ntt . . . 4
- 1.10. starsandbars . . . 4

2. Bits **5**
- 2.1. SoSDP . . . 5
- 2.2. gray . . . 5

3. Miscelanea **5**
- 3.1. Plantilla . . . 5

4. Datastructures **6**
- 4.1. LowestCommonAncestor . . . 6
- 4.2. Mo . . . 6
- 4.3. QueueMin . . . 8
- 4.4. SegmentTree . . . 9
- 4.5. SparseTable . . . 9
- 4.6. UnionFind . . . 10
- 4.7. UnionFind2D . . . 10
- 4.8. fenwickTree . . . 11
- 4.9. lazy . . . 11
- 4.10. sqrt . . . 12
- 4.11. sqrtUpdateinSqrt . . . 12

5. Strings **13**
- 5.1. KMP . . . 13
- 5.2. ahocorasick . . . 13
- 5.3. hashing . . . 17
- 5.4. hashing2d . . . 18
- 5.5. hashingmersense . . . 18
- 5.6. lcs . . . 19
- 5.7. manacher . . . 19

5.8. suffixarray . . . 20
5.9. trie . . . 20
5.10. trie2 . . . 22
5.11. zfunction . . . 22

6. Geometry **22**
6.1. KDTree . . . 22
6.2. Point . . . 24
6.3. convexHull . . . 24
6.4. geometrynotes . . . 25
6.5. iterativequerysegment . . . 26
6.6. laticepoint . . . 26
6.7. nearestVecino . . . 26
6.8. picks_theorem . . . 27
6.9. pointinpoligon . . . 27
6.10. pointintpoligon . . . 28
6.11. twoSegmentIntersec . . . 29

7. Primalidad **30**
7.1. RabinMiller . . . 30
7.2. pollardRho . . . 30

8. Graphs **32**
8.1. 2-sat . . . 32
8.2. 2SAT . . . 33
8.3. Algoritmo_hungaro_de_asignacion . . . 35
8.4. FordFulkerson . . . 35
8.5. HoprofKarp . . . 36
8.6. MinCost-MaxFlow_DEsopo-Pape . . . 36
8.7. MinCostMaxFlow_Dijkstra . . . 37
8.8. articulationpoint . . . 39
8.9. bridges . . . 40
8.10. centroid . . . 40
8.11. componentes_fuertemente_conexos . . . 42
8.12. dijstra . . . 42
8.13. dinic . . . 43
8.14. dinitz . . . 44
8.15. edmonkarp . . . 45
8.16. eulerianpath . . . 45
8.17. floyd_warshall . . . 46
8.18. hld . . . 47
8.19. isDag . . . 48
8.20. isDagv2 . . . 49
8.21. khun . . . 49

8.22. khunMaxbipartiteMatching . . . 50
8.23. lcabinaryliftin . . . 50
8.24. toposort . . . 51

9. Overloads **51**
9.1. pq . . . 51
9.2. set . . . 51

10. Dp **51**
10.1. LCA_DP . . . 51
10.2. covexHullTrick . . . 52
10.3. lis . . . 53
10.4. list . . . 53

1. Math

1.1. Catalan

```
// Catalan, parentesis balanceados, arboles binarios, triangulacion poligono convexto de n + 2 lados, caminos en grilla sin atravesar diagonal
// Cat[n] = C(2n, n) / (n + 1)
// C(n, k) es el coeficiente binomial
Cat[0] = 1;
Cat[n+1] = Cat[n] * 2 * (2 * n + 1) / (n + 2);
Cat[n] = Cat[n-1] * 2 * (2 * n - 1) / (n + 1);

// Convolucion Catalan, parentesis balanceados en k cajas
// Catk[n] = C(2n + k - 1, n) * k / (n + k)
Catk[0] = 1;
Catk[n + 1] = Catk[n] * (2 * n + k) * (2 * n + 1 + k) / (n + 1) / (n + k + 1);
Catk[n] = Catk[n - 1] * (2 * n + k - 1) * (2 * n + k - 2) / n / (n + k);

int main()
{
  // count the number of plane trees with a given set of leaves,
  // the number of ways of inserting parentheses into a sequence,
  // and the number of ways of dissecting a convex polygon into smaller polygons by inserting diagonals
  ll scat[26];//super catalan
  scat[0]=scat[1]=1;
  for(int i=2;i<26;i++)
    scat[i]=(3*(2*i-1)*scat[i-1]-(i-2)*scat[i-2])/(i+1);
  ll cat[26];
  for(int i=0;i<26;i++)
    cat[i]=cata(i);
  return 0;
}
```

1.2. extended

```
int x, y, d;
void extendedEuclid(int a, int b)//ecuacion diofantica ax + by = d
{
  if(b==0) {x=1; y=0; d=a; return;}
  extendedEuclid(b,a%b);
  int x1=y;
  y = x-(a/b)*y;
  x=x1;
}
```

1.3. fastgcd

```
int gcd(int a, int b) {
  if (!a || !b)
    return a | b;
  unsigned shift = __builtin_ctz(a | b);
  a >>= __builtin_ctz(a);
  do {
    b >>= __builtin_ctz(b);
    if (a > b)
      swap(a, b);
    b -= a;
  } while (b);
  return a << shift;
}
```

1.4. fft

```
using cd = complex<double>;
const double PI = acos(-1);

void fft(vector<cd> & a, bool invert) {
    int n = a.size();

    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;

        if (i < j)
```

```
            swap(a[i], a[j]);
    }

    for (int len = 2; len <= n; len <<= 1) {
        double ang = 2 * PI / len * (invert ? -1 :
            1);
        cd wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            cd w(1);
            for (int j = 0; j < len / 2; j++) {
                cd u = a[i+j], v = a[i+j+len/2] * w;
                a[i+j] = u + v;
                a[i+j+len/2] = u - v;
                w *= wlen;
            }
        }
    }

    if (invert) {
        for (cd & x : a)
            x /= n;
    }
}

vector<bool> multiply(vector<int> const& a, vector<
    int> const& b) {
    vector<cd> fa(a.begin(), a.end()), fb(b.begin(),
        b.end());
    int n = 1;
    while (n < a.size() + b.size())
        n <<= 1;

    // n=min(n,int(2e6)+10);
    fa.resize(n);
    fb.resize(n);

    fft(fa, false);
    fft(fb, false);
    for (int i = 0; i < n; i++)
        fa[i] *= fb[i];
    fft(fa, true);

    vector<int> result(n);
    for (int i = 0; i < n; i++)
        result[i] = round(fa[i].real());
    return result;
}
```

1.5. gauss_algebra_lineal

```
const double EPS = 1e-9;
const int INF = 2; // it doesn't actually have to be
    infinity or a big number


int gauss (vector < bitset<N> > a, int n, int m,
    bitset<N> & ans) {
    vector<int> where (m, -1);
    for (int col=0, row=0; col<m && row<n; ++col) {
        for (int i=row; i<n; ++i)
            if (a[i][col]) {
                swap (a[i], a[row]);
                break;
            }
        if (! a[row][col])
            continue;
        where[col] = row;

        for (int i=0; i<n; ++i)
            if (i != row && a[i][col])
                a[i] ^= a[row];
        ++row;
    }
        // The rest of implementation is the same as
            above

    ans.assign (m, 0);
    for (int i=0; i<m; ++i)
        if (where[i] != -1)
            ans[i] = a[where[i]][m] / a[where[i]][i
                ];
    for (int i=0; i<n; ++i) {
        double sum = 0;
        for (int j=0; j<m; ++j)
            sum += ans[j] * a[i][j];
        if (abs (sum - a[i][m]) > EPS)
            return 0;
```

```
    }

    for (int i=0; i<m; ++i)
        if (where[i] == -1)
            return INF;
    return 1;
}
```

1.6. linearsieve

```
std::vector <int> prime;
bool is_composite[MAXN];

void sieve (int n) {
  std::fill (is_composite, is_composite + n, false);
  for (int i = 2; i < n; ++i) {
    if (!is_composite[i]) prime.push_back (i);
    for (int j = 0; j < prime.size () && i * prime[j
       ] < n; ++j) {
      is_composite[i * prime[j]] = true;
      if (i % prime[j] == 0) break;
    }
  }
}

// As is shown in the code, the statement if (i %
   prime[j] == 0) break;
// terminates the loop when p divides i.
// From the analysis above, we can see that the
   inner loop is executed only once for each
   composite.
// Hence, the code above performs in O(n) complexity
   , resulting in its name - the 'linear' sieve.
```

1.7. matrizFibo

```
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;
const ll MOD = 1'000'000'007;

struct Matrix {
    ll mat[2][2];
};

Matrix operator*(const Matrix &a, const Matrix &b){
    Matrix c{};
    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 2; j++) {
            __int128 tmp = 0;
            for (int k = 0; k < 2; k++)
                tmp += (__int128)a.mat[i][k] * b.mat
                   [k][j];
            c.mat[i][j] = (ll)(tmp % MOD);
        }
    return c;
}

Matrix matpow(Matrix base, ll n) {
    Matrix ans{{ {1, 0}, {0, 1} }};
    while (n) {
        if (n & 1) ans = ans * base;
        base = base * base;
        n >>= 1;
    }
    return ans;
}

ll fib(ll n) {
    if (n == 0) return 0;
    Matrix base{{ {1,1}, {1,0} }};
    return matpow(base, n).mat[0][1];
}

int main(){
    ll n;
    cin >> n;
    cout << fib(n) << "\n";
}
```

1.8. multiply

```
vector<int> multiply(vector<int> const& a, vector<
   int> const& b) {
```

```
    vector<cd> fa(a.begin(), a.end()), fb(b.begin(),
        b.end());
    int n = 1;
    while (n < a.size() + b.size())
        n <<= 1;
    fa.resize(n);
    fb.resize(n);

    fft(fa, false);
    fft(fb, false);
    for (int i = 0; i < n; i++)
        fa[i] *= fb[i];
    fft(fa, true);
    // For nnt just return fa
    // and apply MOD to fa[i]
    // If you want return result
    // Clean the right zeros
    vector<int> result(n);
    for (int i = 0; i < n; i++)
        result[i] = round(fa[i].real());

    return result;
}
```

1.9. ntt

```
const int mod = 7340033;
const int root = 5;
const int root_1 = 4404020;
const int root_pw = 1 << 20;
/*
    const int mod = 998244353;
    const int root = 31;
    const int root_1 = 128805723;
    const int root_pw = 1 << 23;

    const int mod = 1004535809;
    const int root = 12289;
    const int root_1 = 313564925;
    const int root_pw = 1 << 21;

    const int mod = 469762049;
    const int root = 2187;
    const int root_1 = 410692747;
    const int root_pw = 1 << 26;
*/
void fft(vector<int> & a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit>>=1) j ^= bit;
        j ^= bit;

        if(i<j) swap(a[i], a[j]);
    }

    for (int len = 2; len <= n; len <<= 1) {
        int wlen = invert ? root_1 : root;
        for (int i = len; i < root_pw; i <<= 1)
            wlen = (int)(1LL * wlen * wlen % mod);

        for (int i = 0; i < n; i += len) {
            int w = 1;
            for (int j = 0; j < len / 2; j++) {
                int u = a[i+j], v = (int)(1LL * a[i+
                    j+len/2] * w % mod);
                a[i+j] = u + v < mod ? u + v : u + v
                     - mod;
                a[i+j+len/2] = u - v >= 0 ? u - v :
                    u - v + mod;
                w = (int)(1LL * w * wlen % mod);
            }
        }
    }
    if (invert) {
        int n_1 = inverse(n, mod);
        for (int & x : a)
            x = (int)(1LL * x * n_1 % mod);
    }
}
```

1.10. starsandbars

```
/**
 * (n+k-1,n)
 *
```

```
 * Si cada caja tiene que tener al menos uno
 * (n-1,k-1)
 *
 *
 */
```

2. Bits

2.1. SoSDP

```
//suma de subsets,donde A[i]tiene la suma de esa
    mascara
for(int i = 0; i<(1<<N); ++i)ans[i] = A[i];
for(int i = 0;i < N; ++i){
    for(int mask = 0; mask < (1<<N); ++mask){
        if(mask & (1<<i))ans[mask] += ans[mask^(1<<i
            )];
    }
}
```

2.2. gray

```
int g (int n) {
    return n ^ (n >> 1);
}
int rev_g (int g) {
  int n = 0;
  for (; g; g >>= 1)
    n ^= g;
  return n;
}
```

3. Miscelanea

3.1. Plantilla

```
#include <bits/stdc++.h>
#include <bits/extc++.h>
using namespace std;
using namespace __gnu_pbds;
typedef long long ll;
typedef long double ld;

typedef tree<pair<ll, ll>, null_type, less<pair<ll,
    ll>>, rb_tree_tag,
    tree_order_statistics_node_update>
    ordered_set_men;
typedef tree<int, null_type, greater<int>,
    rb_tree_tag, tree_order_statistics_node_update>
    ordered_set_may;

#define CRISTIANO_RONALDO_GANO_35_COPAS \
    ios_base::sync_with_stdio(false);   \
    cin.tie(NULL);                      \
    cout.tie(nullptr);
#define hola cout << "hola" << endl;
#define YES cout << "YES" << endl;
#define NO cout << "NO" << endl;
#define dbg(x) cout << #x << ": " << x << endl;
#define dbg2(x, y) cout << #x << ": " << x << " __ "
    << #y << ": " << y << endl;
#define printvii(v_v)                          \
    for (auto [x_x, y_y] : v_v)                \
    {                                          \
        cout << x_x << " " << y_y << endl; \
    }                                          \
    cout << endl;

#define printst(st)          \
    for (auto num : st)      \
        cout << num << " "; \
    cout << endl;
template <typename T>
void printv(T v)
{
    for (auto x : v)
        cout << x << " ";
    cout << endl;
}

#define RAYA cout << "----------------------------"
    << endl;
// #define F first
// #define S second
const ll MOD = 1'000'000'007;
```

```cpp
const vector<int> F = {0, 1, 0, -1};
const vector<int> C = {1, 0, -1, 0};

int main()
{
    CRISTIANO_RONALDO_GANO_35_COPAS
    //sol
    return 0;
}
```

4. Datastructures

4.1. LowestCommonAncestor

```cpp
#include <bits/stdc++.h>

using namespace std;

int n, m;//nodos, links
vector<int> h;//altura del nodo
vector<int> primera;//primera aparicion
// set<int> st;
vector<bool> vis;
vector<vector<int>> gf;
vector<int> nodo;//nodo

void dfs(int currt, int alt)
{
    if (!vis[currt])
    {
        primera[currt] = h.size();
    }
    vis[currt] = 1;
    h.push_back(alt);
    nodo.push_back(currt);

    for (int hijo : gf[currt])
    {
        if (!vis[hijo])
        {
            dfs(hijo, alt + 1);
            h.push_back(alt);
            nodo.push_back(currt);
        }
    }
}

int main()
{
    vis.assign(n, 0);
    primera.assign(n, -1);
    gf.assign(n, vector<int>());
    // st.clear();
    h.clear();
    nodo.clear();
    //0i
    dfs(0, 0);

    //hacer un segmet tree sobre h que es la altura, y responder cout<<nodo[res] donde res es el indice del minimo en el rango, para minimo en rango, donde l y r son los dos nodos
    return 0;
}
```

4.2. Mo

```cpp
struct Query
{
    int l, r, idx;
    Query() {}
    Query(int lx, int rr, int ii)
    {
        l = lx;
        r = rr;
        idx = ii;
    }
};


inline void add(int i)
{
    //hacer algo
}

```

```
inline void remove(int i)
{
    //hacer algo
}

inline bool get_answer()
{
    return imp == 0;
}

inline vector<bool> Mo(vector<Query> &qs)
{
    const int szb = int(ceil(sqrt(N + .0)));
    sort(qs.begin(), qs.end(), [&](const Query &a, const Query &b)
    {
        if (a.l / szb == b.l / szb)
        {
            return ((a.l / szb) & 1) ? a.r < b.r : a.r > b.r;
        }
        return a.l < b.l;
    });
    vector<bool> answers(Q);
    int c_l = 0, c_r = -1;
    for (auto &q : qs)
    {
        int ql = q.l;
        int qr = q.r;
        int qi = q.idx;
        while (c_l > ql)
        {
            c_l--;
            add(c_l);
        }
        while (c_r < qr)
        {
            c_r++;
            add(c_r);
        }
        while (c_l < ql)
        {
            remove(c_l);
            c_l++;
        }
        while (c_r > qr)
        {
            remove(c_r);
            c_r--;
        }
        answers[qi] = get_answer();
    }
    return answers;
}

int main()
{
    CRISTIANO_RONALDO_GANO_35_COPAS;

    int t;
    cin >> t;
    while (t--)
    {
        cin >> N >> Q;

        vec.assign(N, 0);

        for (int i = 0; i < N; i++)
            cin >> vec[i];

        vector<Query> qs(Q);
        for (int i = 0; i < Q; i++)
        {
            int l, r;
            cin >> l >> r;
            l--;
            r--;
            qs[i] = Query(l, r, i);
        }

        ans = 0;
        imp = 0;
        par = 0;
        vector<bool> ans = Mo(qs);

        for (int i = 0; i < Q; i++)
        {
            if (ans[i])
```

```
                cout << "YES\n";
            else
                cout << "NO\n";
        }
        // dbg(mp.size());
        for(int i=0;i<N;i++)mp[vec[i]]=0;
    }

    return 0;
}
```

4.3. QueueMin

```
struct quemin
{
  stack<pair<int,int>> bo, to;
  void push(int n)
  {
    if(bo.empty())
      bo.push(mp(n, n));
    else
      bo.push(mp(n, min(bo.top().s, n)));
  }
  void pop()
  {
    if(to.empty())
    {
      while(!bo.empty())
      {
        if(to.empty())
          to.push(mp(bo.top().f, bo.top().f));
        else
          to.push(mp(bo.top().f, min(bo.top().f, to.
              top().s)));
        bo.pop();
      }
    }
    to.pop();
  }
  int mini()
  {
    int mini = MOD;
    if(!bo.empty())
      mini = bo.top().s;
    if(!to.empty())
      mini = min(mini, to.top().s);
    return mini;
  }
};

struct quemin
{
  pair<int,int> bo[100010], to[100010];
  int boto = -1, toto = -1, ax;
  void push(int n)
  {
    ax = boto + 1;
    if(boto == -1)
      bo[ax] = mp(n, n);
    else
      bo[ax] = mp(n, min(bo[boto].s, n));
    boto++;
  }
  void pop()
  {
    if(toto == -1)
    {
      while(boto > -1)
      {
        ax = toto + 1;
        if(toto == -1)
          to[ax] = mp(bo[boto].f, bo[boto].f);
        else
          to[ax] = mp(bo[boto].f, min(bo[boto].f, to
              [toto].s));
        toto++;
        boto--;
      }
    }
    if(toto > -1)
      toto--;
  }
  int mini()
  {
    int mini = MOD;
    if(boto > -1)
      mini = bo[boto].s;
```

```
    if(toto > -1)
      mini = min(mini, to[toto].s);
    return mini;
  }
};
```

4.4. SegmentTree

```
struct Node
{
    //constructor
    Node operator+(const Node &other)const
    {
        //merge
    }
};

struct SegmentTree
{
    int sz;
    vector<Node> st;

    SegmentTree(int n,vector<ll> &v)
    {
        if(__builtin_popcount(n)!=1)n=(1<<(31-
            __builtin_clz(n)+1));
        sz=n;
        st.assign(2*sz,Node());
        for(int i=0;i<(int)v.size();i++)
        {
            st[sz+i]=Node(v[i]);
        }
        for(int i=2*sz-1;i>=2;i-=2)
        {
            int p=i/2;
            st[p]=st[p*2]+st[p*2+1];//orden
                importanteeeeeee aaaaa media hora
                debuguenado mierda aaaaaaasex0000
        }
    }

    void update(ll num,int pos)//0i
    {
        st[sz+pos]=Node(num);
        pos+=sz;
        while(pos>1)
        {
            int par=pos/2;
            st[par]=st[2*par]+st[2*par+1];
            pos=par;
        }
    }

    Node query(int nodo,int l_n,int r_n,int l_q,int
        r_q)
    {
        //si tengo todo retorna todo
        if(l_q<=l_n and r_n<=r_q)return st[nodo];
        // si no tengo nada retorna neutro
        if(l_q>r_n or l_n>r_q)return Node();//neutro
        int m=(l_n+r_n)/2;
        return query(2*nodo,l_n,m,l_q,r_q)+query(2*
            nodo+1,m+1,r_n,l_q,r_q);
    }

    //(1,0,sz-1,l_q,r_q,val)[0i]s
    int get_first(int v, int tl, int tr, int l, int
        r, int x)
    {
        if(tl > r || tr < l) return -1;
        if(t[v] <= x) return -1;

        if (tl== tr) return tl;

        int tm = tl + (tr-tl)/2;
        int left = get_first(2*v, tl, tm, l, r, x);
        if(left != -1) return left;
        return get_first(2*v+1, tm+1, tr, l ,r, x);
    }
};
```

4.5. SparseTable

```
struct SparseTable {
    int n;
    vector<int> log2;
```

```
    vector<vector<int>> st;

    SparseTable(const vector<int>& a) {
        n = a.size();
        log2.resize(n + 1);
        log2[1] = 0;
        for (int i = 2; i <= n; i++)
            log2[i] = log2[i/2] + 1;

        int k = log2[n] + 1;
        st.assign(n, vector<int>(k));

        for (int i = 0; i < n; i++) st[i][0] = a[i];

        for (int j = 1; j < k; j++)
            for (int i = 0; i + (1 << j) <= n; i++)
                st[i][j] = min(st[i][j-1], st[i + (1
                    << (j-1))][j-1]);
    }

    // Consulta de minimo en el rango [l, r]oi
    int query(int l, int r) {
        int j = log2[r - l + 1];
        return min(st[l][j], st[r - (1 << j) + 1][j
           ]);
    }
};
```

4.6. UnionFind

```
struct unionFind {
  vi p;
  unionFind(int n) : p(n, -1) {}
  int findParent(int v) {
    if (p[v] == -1) return v;
    return p[v] = findParent(p[v]);
  }
  bool join(int a, int b) {
    a = findParent(a);
    b = findParent(b);
    if (a == b) return false;
    p[a] = b;
    return true;
  }
};
```

4.7. UnionFind2D

```
const int MAX=1000000+10;
vector<int> sz(MAX,1);
vector<int> par(MAX,-1);
int n,m;

int comp=0;


int id(int x,int y)
{
    return x*m+y;
}

int find(int v)
{
    if(v==par[v]) return v;
    return par[v]=find(par[v]);
}

int find(int x,int y)
{
    return find(id(x,y));
}

void unite(int x1,int y1,int x2,int y2)
{
    int a=find(x1,y1);
    int b=find(x2,y2);

    if(a!=b)
    {
        if(sz[a]<sz[b]) swap(a,b);

        par[b]=a;
        sz[a]+=sz[b];
    }
}

```

```
bool same(int x1,int y1,int x2,int y2)
{
    return find(x1,y1)==find(x2,y2);
}

int size(int x,int y)
{
    return sz[find(x,y)];
}
```

4.8. fenwickTree

```
#include <bits/stdc++.h>

using namespace std;
typedef long long ll;


struct Bit
{
    private:
        vector<ll> bit;
        int sz;
    public:

    Bit(int n){bit.resize(n+1,0);this->sz=n;}

    void init(vector<ll> vec)
    {
        for(int i=0;i<sz;i++)
        {
            update(i+1,vec[i]);
        }
    }

    void update(int p,ll val)//1i
    {
        while(p<=sz)
        {
            bit[p]+=val;
            p+=(p&(-p));
        }
    }
    ll query(int p)//[1,p]1i
    {
        ll ans=0;
        while(p>0)
        {
            ans+=bit[p];
            p-=(p&-p);
        }
        return ans;
    }

    ll rquery(ll left,ll right)//[l,r]1i
    {
        return query(right)-query(left-1);
    }
};
```

4.9. lazy

```
struct SegmentTreeLazy
{
    void push(int nodo,int l_n, int r_n){

        /**
         * 1.Aplicar lazy del padre(nodo) a los
            hijos
         * 2.marcar el lazy a los hijos
         * 3.matar el lazy del padre */

    }

    long long query(int nodo, int l_n,int r_n,int
       l_q,int r_q){
        if(l_q<=l_n and r_n<=r_q) return st[nodo].sc
           ;
        if(l_q>r_n or l_n>r_q) return 0;
        push(nodo,l_n,r_n);
        int m=(l_n+r_n)>>1;
        return (long long)((query(2*nodo,l_n,m,l_q,
           r_q)+query(2*nodo+1,m+1,r_n,l_q,r_q)));
    }
    void update(int nodo, int l_n,int r_n,int l_q,
       int r_q,ll val){
```

```
        if(l_q>r_n or l_n>r_q) return;
        if(l_q<=l_n and r_n<=r_q){
            //aplicar el lazy, osea si fuera suma
               sumarlo al nodo y guardar el lazy
            st[nodo].lazy_x+=val;
            return;
        }
        else{
            push(nodo,l_n,r_n);
            int m=(l_n+r_n)/2;
            update(2*nodo,l_n,m,l_q,r_q,val);
            update(2*nodo+1,m+1,r_n,l_q,r_q,val);
            st[nodo]=st[nodo*2]+st[nodo*2+1];
        }
    }
};
```

4.10. sqrt

```
struct Sqrt
{
    vector<ll> vec;
    vector<ll> blocks;
    int sz;
    int szb;

    Sqrt(vector<ll> v,int n)
    {
        sz=n;
        szb=int(ceil(sqrt(n)));
        vec=v;
        blocks.assign(szb,0ll);
        for(int i=0;i<sz;i++)
        {
            blocks[i/szb]+=vec[i];
        }
    }

    ll q(int r)//[0,r)
    {
        ll ans=0;
        for(int i=0;i<r/szb;i++)
        {
            ans+=blocks[i];
        }
        for(int i=(r/szb)*szb;i<r;i++)
        {
            ans+=vec[i];
        }
        return ans;
    }

    ll rq(int l,int r)//[l,r]1i
    {
        return q(r)-q(l-1);
    }

    void update(int pos,ll v)//0i
    {
        blocks[pos/szb]-=vec[pos];
        vec[pos]=v;
        blocks[pos/szb]+=vec[pos];
    }
};
```

4.11. sqrtUpdateinSqrt

```
#include <bits/stdc++.h>
typedef long long ll;
using namespace std;

struct Sqrt
{
    vector<ll> vec;
    vector<ll> blocks;
    int sz;
    int szb;

    Sqrt(vector<ll> v, int n)
    {
        sz = n;
        szb = int(ceil(sqrt(n)));
        vec = v;
        blocks.assign(szb, 1e18);
        for (int i = 0; i < sz; i++)
        {
```

```
            blocks[i / szb]=min(vec[i],blocks[i / szb]);
        }
    }

    ll rq(int l, int r) //[l,r]0i
    {
        ll ans=1e18;
        int c_l = l / szb, c_r = r / szb;
        if (c_l == c_r)
            for (int i = l; i <= r; i++)
                ans=min(ans,vec[i]);
        else
        {
            for (int i = l, end = (c_l + 1) * szb - 1; i <= end; i++)
                ans=min(ans,vec[i]);
            for (int i = c_l + 1; i <= c_r - 1; i++)
                ans=min(ans,blocks[i]);
            for (int i = c_r * szb; i <= r; i++)
                ans=min(ans,vec[i]);
        }
        return ans;
    }

    void update(int pos, ll v) // 0i
    {
        vec[pos] = v;
        ll ans=1e18;

        int ini=(pos/szb)*szb;
        for(int i=ini;i<min(ini+szb,sz);i++)
        {
            ans=min(ans,vec[i]);
        }
        blocks[pos/szb]=ans;
    }
};
```

5. Strings

5.1. KMP

```
vector<int> prefix_function(string s)
{
    int n = (int)s.length();
    vector<int> pi(n);
    for (int i = 1; i < n; i++)
    {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j])
            j = pi[j - 1];
        if (s[i] == s[j])
            j++;
        pi[i] = j;
    }
    return pi;
}

inline void solve()
{
    string texto, sub;
    cin >> texto >> sub;
    int n = texto.size();
    int m = sub.size();
    vector<int> vec = prefix_function(sub + "$" + texto);
    cout << count(vec.begin(), vec.end(), m) << endl;
}
```

5.2. ahocorasick

```
#include <bits/stdc++.h>
using namespace std;

struct AhoCorasick {
    static constexpr int ALPHA = 26;
    static constexpr char BASE = 'a';

    struct Node {
        array<int, ALPHA> next;
        int link;
        int outputLink;
        vector<int> output;

```

```
        // Complejidad: O(ALPHA)
        Node() : link(0), outputLink(-1) {
            next.fill(-1);
        }
    };

    vector<Node> trie;
    vector<int> patternLength;
    bool built;

    // Complejidad: O(ALPHA)
    AhoCorasick() {
        init();
    }

    // Reinicia toda la estructura.
    // Complejidad: O(total de nodos)
    void init() {
        trie.clear();
        trie.emplace_back();
        patternLength.clear();
        built = false;
    }

    // Convierte un caracter valido en su indice.
    // Complejidad: O(1)
    int charId(char c) const {
        return c - BASE;
    }

    // Verifica si el caracter pertenece al alfabeto
    .
    // Complejidad: O(1)
    bool validChar(char c) const {
        int id = charId(c);
        return 0 <= id && id < ALPHA;
    }

    // Agrega un patron y retorna su identificador.
    // Los identificadores se asignan desde cero.
    // Complejidad: O(longitud del patron)
    int addPattern(const string& pattern) {
        int patternId = (int)patternLength.size();
        patternLength.push_back((int)pattern.size())
        ;

        int node = 0;

        for (char c : pattern) {
            int id = charId(c);

            assert(0 <= id && id < ALPHA);

            if (trie[node].next[id] == -1) {
                trie[node].next[id] = (int)trie.size
                    ();
                trie.emplace_back();
            }

            node = trie[node].next[id];
        }

        trie[node].output.push_back(patternId);
        built = false;

        return patternId;
    }

    // Construye failure links, output links y
    transiciones completas.
    // Debe ejecutarse despues de agregar todos los
    patrones.
    // Complejidad: O(numero de nodos * ALPHA)
    void build() {
        queue<int> q;

        trie[0].link = 0;
        trie[0].outputLink = -1;

        for (int c = 0; c < ALPHA; ++c) {
            int child = trie[0].next[c];

            if (child == -1) {
                trie[0].next[c] = 0;
            } else {
                trie[child].link = 0;
                trie[child].outputLink = trie[0].
                    output.empty() ? -1 : 0;
```

```
                q.push(child);
            }
        }

        while (!q.empty()) {
            int node = q.front();
            q.pop();

            for (int c = 0; c < ALPHA; ++c) {
                int child = trie[node].next[c];

                if (child == -1) {
                    trie[node].next[c] = trie[trie[
                        node].link].next[c];
                    continue;
                }

                int failure = trie[trie[node].link].
                    next[c];
                trie[child].link = failure;

                if (!trie[failure].output.empty()) {
                    trie[child].outputLink = failure
                        ;
                } else {
                    trie[child].outputLink = trie[
                        failure].outputLink;
                }

                q.push(child);
            }
        }

        built = true;
    }

    // Retorna el siguiente estado al procesar un
        caracter.
    // Requiere que build haya sido ejecutado.
    // Complejidad: O(1)
    int advance(int state, char c) const {
        assert(built);

        if (!validChar(c)) {
            return 0;
        }

        return trie[state].next[charId(c)];
    }

    // Ejecuta una funcion para cada patron que
        termina en el estado actual.
    // callback recibe patternId.
    // Complejidad: O(numero de coincidencias
        reportadas + cadena de outputs)
    template <class Callback>
    void forEachOutput(int state, Callback callback)
        const {
        for (int node = state; node != -1; node =
            trie[node].outputLink) {
            for (int patternId : trie[node].output)
                {
                callback(patternId);
            }
        }
    }

    // Retorna todas las coincidencias como pares:
    // {posicion inicial, patternId}.
    // Complejidad: O(longitud del texto + numero de
        coincidencias)
    vector<pair<int, int>> findAll(const string&
        text) const {
        assert(built);

        vector<pair<int, int>> matches;
        int state = 0;

        for (int i = 0; i < (int)text.size(); ++i) {
            state = advance(state, text[i]);

            forEachOutput(state, int patternId {
                int start = i - patternLength[
                    patternId] + 1;
                matches.push_back({start, patternId
                    });
            });
        }
```

```
        return matches;
    }

    // Retorna cuantas veces aparece cada patron en el texto.
    // Cuenta apariciones superpuestas.
    // Complejidad: O(longitud del texto + numero de coincidencias)
    vector<long long> countOccurrences(const string& text) const {
        assert(built);

        vector<long long> count(patternLength.size(), 0);
        int state = 0;

        for (char c : text) {
            state = advance(state, c);

            forEachOutput(state, int patternId {
                ++count[patternId];
            });
        }

        return count;
    }

    // Retorna la primera posicion de cada patron.
    // Retorna -1 para patrones que no aparecen.
    // Complejidad: O(longitud del texto + numero de coincidencias)
    vector<int> firstOccurrence(const string& text) const {
        assert(built);

        vector<int> first(patternLength.size(), -1);
        int state = 0;

        for (int i = 0; i < (int)text.size(); ++i) {
            state = advance(state, text[i]);

            forEachOutput(state, int patternId {
                if (first[patternId] == -1) {
                    first[patternId] = i - patternLength[patternId] + 1;
                }
            });
        }

        return first;
    }

    // Verifica si al menos un patron aparece en el texto.
    // Complejidad: O(longitud del texto) en el peor caso
    bool containsAny(const string& text) const {
        assert(built);

        int state = 0;

        for (char c : text) {
            state = advance(state, c);

            if (!trie[state].output.empty() || trie[state].outputLink != -1) {
                return true;
            }
        }

        return false;
    }

    // Verifica si un patron especifico aparece en el texto.
    // Complejidad: O(longitud del texto + numero de coincidencias)
    bool containsPattern(const string& text, int targetId) const {
        assert(built);
        assert(0 <= targetId && targetId < (int)patternLength.size());

        int state = 0;

        for (char c : text) {
            state = advance(state, c);
```

```
            bool found = false;

            forEachOutput(state, int patternId {
                if (patternId == targetId) {
                    found = true;
                }
            });

            if (found) {
                return true;
            }
        }

        return false;
    }

    // Retorna la cantidad de nodos del automata.
    // Complejidad: O(1)
    int nodeCount() const {
        return (int)trie.size();
    }

    // Retorna la cantidad de patrones agregados.
    // Complejidad: O(1)
    int patternCount() const {
        return (int)patternLength.size();
    }
};
```

5.3. hashing

```
struct StrHash
{ // Hash polinomial con exponentes decrecientes.
    static constexpr ll ms[] = {1'000'000'007, 1'000
       '000'403};
    static constexpr ll b = 500'000'000;
    vector<ll> hs[2], bs[2];
    StrHash(string const &s)
    {
        int n = s.length();
        for(int k=0;k<2;k++)
        {
            hs[k].resize(n + 1), bs[k].resize(n + 1,
                1);
            for(int i=0;i<n;i++)
            {
                hs[k][i + 1] = (hs[k][i] * b + s[i])
                    % ms[k];
                bs[k][i + 1] = bs[k][i] * b % ms[k];
            }
        }
    }
    ll get(int idx, int len) const
    { // Hashes en `s[idx, idx+len)`.
        ll h[2];
        for(int k=0;k<2;k++)
        {
            h[k] = hs[k][idx + len] - hs[k][idx] *
               bs[k][len] % ms[k];
            if (h[k] < 0)
                h[k] += ms[k];
        }
        return (h[0] << 32) | h[1];
    }
};

//concate substrings(or strings) from non necesary
   two differents strings [idx,indx+lex)
ll concat_cross_hashes(const StrHash& A, int i1, int
    len1, const StrHash& B, int i2, int len2)
{
    ll res[2];
    for (int k = 0; k < 2; ++k)
    {
        // hash de substring A[i1..i1+len1-1]
        ll h1 = A.hs[k][i1 + len1] - A.hs[k][i1] * A
            .bs[k][len1] % A.ms[k];
        if (h1 < 0) h1 += A.ms[k];

        // hash de substring B[i2..i2+len2-1]
        ll h2 = B.hs[k][i2 + len2] - B.hs[k][i2] * B
            .bs[k][len2] % B.ms[k];
        if (h2 < 0) h2 += B.ms[k];

        // combinacion: h1 * b^len2 + h2
        res[k] = (h1 * A.bs[k][len2] + h2) % A.ms[k
```

```
            ];
    }
    return (res[0] << 32) | res[1];
}


//0 indexed
ll h=StrHash("Hola").get(0,0+"Hola".size());
```

5.4. hashing2d

```
struct Hashing
{
    vector<vector<int>> hs;
    vector<int> PWX, PWY;
    int n, m;
    static const int PX = 3731, PY = 2999, mod =
        998244353;
    Hashing() {}
    Hashing(vector<string> &s)
    {
        n = (int)s.size(), m = (int)s[0].size();
        hs.assign(n + 1, vector<int>(m + 1, 0));
        PWX.assign(n + 1, 1);
        PWY.assign(m + 1, 1);
        for (int i = 0; i < n; i++)
            PWX[i + 1] = 1LL * PWX[i] * PX % mod;
        for (int i = 0; i < m; i++)
            PWY[i + 1] = 1LL * PWY[i] * PY % mod;
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < m; j++)
            {
                hs[i + 1][j + 1] = s[i][j] - 'a' +
                    1;
            }
        }
        for (int i = 0; i <= n; i++)
        {
            for (int j = 0; j < m; j++)
            {
                hs[i][j + 1] = (hs[i][j + 1] + 1LL *
                    hs[i][j] * PY % mod) % mod;
            }
        }
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j <= m; j++)
            {
                hs[i + 1][j] = (hs[i + 1][j] + 1LL *
                    hs[i][j] * PX % mod) % mod;
            }
        }
    }
    int get_hash(int x1, int y1, int x2, int y2)
    { // 1-indexed
        assert(1 <= x1 && x1 <= x2 && x2 <= n);
        assert(1 <= y1 && y1 <= y2 && y2 <= m);
        x1--;
        y1--;
        int dx = x2 - x1, dy = y2 - y1;
        return (1LL * (hs[x2][y2] - 1LL * hs[x2][y1]
            * PWY[dy] % mod + mod) % mod -
                1LL * (hs[x1][y2] - 1LL * hs[x1][y1]
                    * PWY[dy] % mod + mod) % mod *
                PWX[dx] % mod + mod) %
            mod;
    }
    int get_hash()
    {
        return get_hash(1, 1, n, m);
    }
};
```

5.5. hashingmersense

```
#include <bits/stdc++.h>
using namespace std;

struct StrHash {
    using ull = unsigned long long;

    static const ull MOD = (1ULL << 61) - 1;

    // Generacion de base aleatoria segura
    static ull get_base() {
```

```
        static ull B = 0;
        if (B == 0) {
            mt19937_64 rng(chrono::steady_clock::now
                ().time_since_epoch().count());
            B = rng() % (MOD - 300) + 256;
            if (B % 2 == 0) B++; // Es mejor que la
                base sea impar
        }
        return B;
    }

    vector<ull> h, p;

    // Multiplicacion modular rapida O(1) sin %
    static inline ull mul(ull a, ull b) {
        unsigned __int128 prod = (unsigned __int128)
            a * b;
        ull r = (ull)(prod & MOD) + (ull)(prod >>
            61);
        if (r >= MOD) r -= MOD;
        return r;
    }

    // Suma modular rapida
    static inline ull add(ull a, ull b) {
        ull res = a + b;
        if (res >= MOD) res -= MOD;
        return res;
    }

    // Resta modular rapida
    static inline ull sub(ull a, ull b) {
        ull res = a - b;
        if (a < b) res += MOD;
        return res;
    }

    StrHash(const string& s) {
        int n = s.size();
        h.resize(n + 1);
        p.resize(n + 1);
        p[0] = 1;

        ull B = get_base();

        for (int i = 0; i < n; i++) {
            h[i + 1] = add(mul(h[i], B), s[i]);
            p[i + 1] = mul(p[i], B);
        }
    }

    ull get(int l, int len) const {
        return sub(h[l + len], mul(h[l], p[len]));
    }
};
```

5.6. lcs

```
int longestCommonSubsequence(string text1, string
    text2) {
  int n = text1.size(), m = text2.size();
  vector<vector<int>> lcs(n+1, vector<int>(m+1));
  for(int i = 1; i <= n; i++)
  for(int j = 1; j <= m; j++)
  {
    lcs[i][j] = max(lcs[i - 1][j], lcs[i][j - 1]);
    if(text1[i - 1] == text2[j - 1])
      lcs[i][j] = max(lcs[i][j], lcs[i - 1][j - 1] +
          1);
  }
  return lcs[n][m];
}
```

5.7. manacher

```
vector<int> manacher_odd(string s) {
    int n = s.size();
    s = "$" + s + "^";
    vector<int> p(n + 2);
    int l = 0, r = 1;
    for(int i = 1; i <= n; i++) {
        if(i <= r) {
            p[i] = min(r - i, p[l + (r - i)]);
        }
        while(s[i - p[i]] == s[i + p[i]]) {
            p[i]++;
```

```
        }
        if(i + p[i] > r) {
            l = i - p[i], r = i + p[i];
        }
    }
    return vector<int>(begin(p) + 1, end(p) - 1);
}

vector<int> manacher(string s) {
    string t;
    for(auto c: s) {
        t += string("#") + c;
    }
    auto res = manacher_odd(t + "#");
    return vector<int>(begin(res) + 1, end(res) - 1)
        ;
}
```

5.8. suffixarray

```
vector<int> suffixa(string &s){
  int n = s.size(), cc, ax;
  vector<int> sa(n), sa1(n), col(n), col1(n), head(n
    );
  fore(i, 0, n) sa[i] = i;
  auto cmp = [&](int a, int b){ return s[a] < s[b];
    };
  stable_sort(sa.begin(), sa.end(), cmp);
  head[0] = col[sa[0]] = cc = 0;
  fore(i, 1, n){
    if(s[sa[i]] != s[sa[i-1]])
      cc++, head[cc] = i;
    col[sa[i]] = cc;
  }
  for(int k = 1; k < n; k *= 2){
    fore(i, 0, n){
      ax = (sa[i] - k + n) % n;
      sa1[head[col[ax]]++] = ax;
    }
    swap(sa, sa1);
    col1[sa[0]] = head[0] = cc = 0;
    fore(i, 1, n){
      if(col[sa[i]] != col[sa[i - 1]] || col[(sa[i]
        + k) % n] != col[(sa[i - 1] + k) % n])
        cc++, head[cc] = i;
      col1[sa[i]] = cc;
    }
    swap(col, col1);
    if(col[sa[n - 1]] == n - 1) break;
  }
  return sa;
}
vector<int> lcp(string &s, vector<int> &sa){
  int n = s.size(), k, z = 0;
  vector<int> sa1(n), lcp(n);
  fore(i, 0, n) sa1[sa[i]] = i;
  fore(i, 0, n){
    k = sa1[i];
    if(k < n - 1)
      while(s[i + z] == s[sa[k+1] + z])
        z++;
    lcp[k] = z; z = max(z-1, 0ll);
  }
  return lcp;
}
```

5.9. trie

```
#include <bits/stdc++.h>
using namespace std;

struct Trie {
    static constexpr int ALPHABET = 26;

    struct Node {
        int next[ALPHABET];
        int prefixCount;
        int wordCount;

        Node() {
            fill(next, next + ALPHABET, -1);
            prefixCount = 0;
            wordCount = 0;
        }
    };

```

```
    vector<Node> trie;

    Trie(int maxNodes = 1) {
        trie.reserve(maxNodes);
        trie.emplace_back(); // Nodo 0: raiz
    }

    // Reinicia completamente el Trie
    void clear() {
        trie.clear();
        trie.emplace_back();
    }

    // Crea un nuevo nodo y devuelve su indice
    int newNode() {
        trie.emplace_back();
        return (int)trie.size() - 1;
    }

    // Inserta una palabra
    // Supone que contiene solamente letras entre 'a' y 'z'
    void insert(const string& word) {
        int node = 0;
        trie[node].prefixCount++;

        for (char ch : word) {
            int c = ch - 'a';

            if (trie[node].next[c] == -1) {
                trie[node].next[c] = newNode();
            }

            node = trie[node].next[c];
            trie[node].prefixCount++;
        }

        trie[node].wordCount++;
    }

    // Devuelve el nodo donde termina la cadena
    // Devuelve -1 si la cadena no existe como camino en el Trie
    int findNode(const string& word) const {
        int node = 0;

        for (char ch : word) {
            int c = ch - 'a';

            if (c < 0 || c >= ALPHABET)
                return -1;

            if (trie[node].next[c] == -1)
                return -1;

            node = trie[node].next[c];
        }

        return node;
    }

    // Devuelve cuantas veces se inserto exactamente una palabra
    int countWord(const string& word) const {
        int node = findNode(word);

        if (node == -1)
            return 0;

        return trie[node].wordCount;
    }

    // Indica si una palabra fue insertada al menos una vez
    bool contains(const string& word) const {
        return countWord(word) > 0;
    }

    // Devuelve cuantas palabras insertadas comienzan con prefix
    int countPrefix(const string& prefix) const {
        int node = findNode(prefix);

        if (node == -1)
            return 0;

        return trie[node].prefixCount;
    }
```

```
    // Elimina una aparicion de la palabra
    // Devuelve false si la palabra no estaba insertada
    bool erase(const string& word) {
        if (!contains(word))
            return false;

        int node = 0;
        trie[node].prefixCount--;

        for (char ch : word) {
            int c = ch - 'a';
            node = trie[node].next[c];
            trie[node].prefixCount--;
        }

        trie[node].wordCount--;
        return true;
    }

    // Devuelve la cantidad de nodos utilizados
    int size() const {
        return (int)trie.size();
    }
};
```

5.10. trie2

```
const int K = 26;

struct Vertex {
    int next[K];
    bool output = false;

    Vertex() {
        fill(begin(next), end(next), -1);
    }
};

vector<Vertex> trie(1);

void add_string(string const& s) {
    int v = 0;
    for (char ch : s) {
        int c = ch - 'a';
        if (trie[v].next[c] == -1) {
            trie[v].next[c] = trie.size();
            trie.emplace_back();
        }
        v = trie[v].next[c];
    }
    trie[v].output = true;
}
```

5.11. zfunction

```
vector<int> z_function(string s) {
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for(int i = 1; i < n; i++) {
        if(i < r) {
            z[i] = min(r - i, z[i - l]);
        }
        while(i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }
        if(i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}
```

6. Geometry

6.1. KDTree

```
double mejordist;
double menordis;

//P x,y
```

```
//dist ditancia euclidana
//El kde tree nos puede ayudar a calcular el vecino mas cercano a un punto

struct KDTree
{
    struct Nodo
    {
        P p;
        Nodo* left;
        Nodo* rigth;
        int id;
        Nodo(P pt):p(pt),left(nullptr),rigth(nullptr){}
    };

    Nodo* raiz;


    Nodo* insertar(Nodo* nodo,P pt,int dep)
    {
        if(nodo==nullptr)return new Nodo(pt);
        int dim=dep%2;
        if(dim==0)
        {
            if(pt.x<nodo->p.x)nodo->left=insertar(nodo->left,pt,dep+1);
            else nodo->rigth=insertar(nodo->rigth,pt,dep+1);
        }
        else
        {
            if(pt.y<nodo->p.y)nodo->left=insertar(nodo->left,pt,dep+1);
            else nodo->rigth=insertar(nodo->rigth,pt,dep+1);
        }
        return nodo;
    }

    // void search(Nodo *nodo,P pt,int dep)
    // {
    //     if(nodo==nullptr)return;
    //     ans=nodo->p;
    //     int dim=dep%2;

    //     if(dim==0)
    //     {
    //         if(pt.x<nodo->p.x)search(nodo->left,pt,dep+1);
    //         else search(nodo->rigth,pt,dep+1);
    //     }
    //     else
    //     {
    //         if(pt.y<nodo->p.y)search(nodo->left,pt,dep+1);
    //         else search(nodo->rigth,pt,dep+1);
    //     }


    // }

    void search(Nodo* nodo,P pt,int dep,int mn=0)
    {
        if(nodo==nullptr)return;

        double d=dist(nodo->p,pt);

        if(d>mn and d<mejordist)
        {
            mejordist=d;
            ans=nodo->p;
        }

        int dim=dep%2;

        Nodo* first;
        Nodo* second;

        if((dim==0 && pt.x<nodo->p.x) || (dim==1 && pt.y<nodo->p.y))
        {
            first=nodo->left;
            second=nodo->rigth;
        }
        else
        {
            first=nodo->rigth;
```

```
            second=nodo->left;
        }

        search(first,pt,dep+1,mn);

        double planeDist;

        if(dim==0)planeDist=(pt.x-nodo->p.x)*(pt.x-nodo->p.x);
        else planeDist=(pt.y-nodo->p.y)*(pt.y-nodo->p.y);
        if(planeDist<mejordist)search(second,pt,dep+1,mn);
    }

    KDTree():raiz(nullptr){};

};
```

6.2. Point

```
/*typedef double T;
typedef complex<T> pt;
#define x real()
#define y imag()*/

//typedef long long ll;
//typedef long double ll;

struct point
{
  ll x, y;
  point() {}
  point(ll x, ll y): x(x), y(y) {}
  point operator -(point p) {return point(x - p.x, y - p.y);}
  point operator +(point p) {return point(x + p.x, y + p.y);}
  ll sq() {return x * x + y * y;}
  double abs() {return sqrt(sq());}
  ll operator ^(point p) {return x * p.y - y * p.x;}
    ll operator *(point p) {return x * p.x + y * p.y;}
    point operator *(ll a) {return point(x * a, y * a);}
  bool operator <(const point& p) const {return x == p.x ? y < p.y : x < p.x;}
  bool left(point a, point b) {return ((b - a) ^ (*this - a)) >= 0;}
  ostream& operator<<(ostream& os) {
    return os << "("<< x << "," << y << ")";
  }

};

void polarSort(vector<point>& v) {
  sort(v.begin(), v.end(), [] (point a, point b) {
    const point origin{0, 0};
    bool ba = a < origin, bb = b < origin;
    if (ba != bb) { return ba < bb; }
    return (a^b) > 0;
  });
}
```

6.3. convexHull

```
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

struct pt
{
  double x, y;
  bool operator==(pt const &t) const
  {
    return x == t.x && y == t.y;
  }
};

int orientation(pt a, pt b, pt c)
{
  double v = a.x * (b.y - c.y) + b.x * (c.y - a.y) + c.x * (a.y - b.y);
  if (v < 0)
    return -1; // clockwise
```

```
  if (v > 0)
    return +1; // counter-clockwise
  return 0;
}

bool cw(pt a, pt b, pt c, bool include_collinear)
{
  int o = orientation(a, b, c);
  return o < 0 || (include_collinear && o == 0);
}
bool collinear(pt a, pt b, pt c) { return
    orientation(a, b, c) == 0; }

void convex_hull(vector<pt> &a, bool
    include_collinear = false)
{
  pt p0 = *min_element(a.begin(), a.end(), [](pt a,
      pt b)
             { return make_pair(a.y, a.x) <
                make_pair(b.y, b.x); });
  sort(a.begin(), a.end(), [&p0](const pt &a, const
      pt &b)
      {
         int o = orientation(p0, a, b);
         if (o == 0)
             return (p0.x-a.x)*(p0.x-a.x) + (p0.y-a.y
                )*(p0.y-a.y)
                 < (p0.x-b.x)*(p0.x-b.x) + (p0.y-b.y)
                    *(p0.y-b.y);
         return o < 0; });
  if (include_collinear)
  {
    int i = (int)a.size() - 1;
    while (i >= 0 && collinear(p0, a[i], a.back()))
      i--;
    reverse(a.begin() + i + 1, a.end());
  }

  vector<pt> st;
  for (int i = 0; i < (int)a.size(); i++)
  {
    while (st.size() > 1 && !cw(st[st.size() - 2],
        st.back(), a[i], include_collinear))
      st.pop_back();
    st.push_back(a[i]);
  }

  if (include_collinear == false && st.size() == 2
      && st[0] == st[1])
    st.pop_back();

  a = st;
}

vector<pt> a;

int main()
{

  int n;cin>>n;
  a.resize(n);
  //leer como puntos
  convex_hull(a,true);//true incluye colinear, false
      no lo hace

}
```

6.4. geometrynotes

```
/**
 *
 *
 * Paralelas:           ax * by - ay * bx == 0
 * Perpendiculares:     ax * bx + ay * by == 0
 *
 *
 * Ecuacion punto-pendiente:
 * y - y1 = m * (x - x1)
 *
 * Pendiente:
 * m = (y2 - y1) / (x2 - x1)
 *
 * Si trabajas con pendientes:
 * m1 == m2        -> paralelas
 * m1 * m2 == -1   -> perpendiculares
 *
 */
```

6.5. iterativequerysegment

```
int query(int l, int r)
{
    int ans = INT_MIN;

    for(l += n, r += n; l <= r; l >>= 1, r >>= 1)
    {
        if(l & 1) ans = max(ans, st[l++]);
        if(!(r & 1)) ans = max(ans, st[r--]);
    }

    return ans;
}
```

6.6. laticepoint

```
//calculates number of integer points (x ; y) such
    for 0 <= x < n and 0 < y <= piso(k * x + b)
int count_lattices(double k, double b, long long n)
    {
    auto fk = floor(k);
    auto fb = floor(b);
    auto cnt = 0LL;
    if (k > 1 - EPS || b > 1 -EPS) {
        cnt += (fk * (n - 1) + 2 * fb) * n / 2;
        k -= fk;
        b -= fb;
    }
    auto t = k * n + b;
    auto ft = floor(t);
    if (ft > 1 - EPS) {
        cnt += count_lattices(1 / k, (t - floor(t))
           / k, floor(t));
    }
    return cnt;
}
pair<double, double> kb(point a, point b)
{
  return mp((b.y - a.y) / (b.x - a.x), (a.y * b.x -
      a.x * b.y) / (b.x - a.x));
}

// # of lattice points s.t. ax+by<=c, x,y>0 (a,b is
    positive integer)
ll f(ll a, ll b, ll c){
  if(c<=0) return 0;
  if(a<b) swap(a, b);
  ll m=c/a;
  if(a==b) return m*(m-1)/2;
  ll k=(a-1)/b, h=(c-a*m)/b;
  return f(b,a-b*k,c-b*(k*m+h))+k*m*(m-1)/2+m*h;
}

// # of lattice points s.t. ax+by<=c, 0<x<=X, 0<y<=Y
     (a,b is positive integer)
ll g(ll a, ll b, ll c, ll X, ll Y){
  if(a*X+b*Y<=c) return X*Y;
  return f(a,b,c)-f(a,b,c-a*X)-f(a,b,c-b*Y)+f(a,b,c-
      a*X-b*Y);
}
```

6.7. nearestVecino

```
// Closest Neighbor - O(n * log(n))
const ll N = 1e6+3, INF = 1e18;
ll n, cn[N], x[N], y[N]; // number of points, closes
    neighbor, x coordinates, y coordinates

ll sqr(ll i) { return i*i; }
ll dist(int i, int j) { return sqr(x[i]-x[j]) + sqr(
    y[i]-y[j]); }
ll dist(int i) { return i == cn[i] ? INF : dist(i,
    cn[i]); }

bool cpx(int i, int j) { return x[i] < x[j] or (x[i]
     == x[j] and y[i] < y[j]); }
bool cpy(int i, int j) { return y[i] < y[j] or (y[i]
     == y[j] and x[i] < x[j]); }

ll calc(int i, ll x0) {
  ll dlt = dist(i) - sqr(x[i]-x0);
  return dlt >= 0 ? ceil(sqrt(dlt)) : -1;
}

void updt(int i, int j, ll x0, ll &dlt) {
```

```
  if (dist(i) > dist(i, j)) cn[i] = j, dlt = calc(i,
      x0);
}

void cmp(vi &u, vi &v, ll x0) {
  for(int a=0, b=0; a<u.size(); ++a) {
    ll i = u[a], dlt = calc(i, x0);
    while(b < v.size() and y[i] > y[v[b]]) b++;
    for(int j = b-1; j >= 0     and y[i] - dlt <= y[
        v[j]]; j--) updt(i, v[j], x0, dlt);
    for(int j = b; j < v.size() and y[i] + dlt >= y[
        v[j]]; j++) updt(i, v[j], x0, dlt);
  }
}

void slv(vi &ix, vi &iy) {
  int n = ix.size();
  if (n == 1) { cn[ix[0]] = ix[0]; return; }

  int m = ix[n/2];

  vi ix1, ix2, iy1, iy2;
  for(int i=0; i<n; ++i) {
    if (cpx(ix[i], m)) ix1.push_back(ix[i]);
    else ix2.push_back(ix[i]);

    if (cpx(iy[i], m)) iy1.push_back(iy[i]);
    else iy2.push_back(iy[i]);
  }

  slv(ix1, iy1);
  slv(ix2, iy2);

  cmp(iy1, iy2, x[m]);
  cmp(iy2, iy1, x[m]);
}

void slv(int n) {
  vi ix, iy;
  ix.resize(n);
  iy.resize(n);
  for(int i=0; i<n; ++i) ix[i] = iy[i] = i;
  sort(ix.begin(), ix.end(), cpx);
  sort(iy.begin(), iy.end(), cpy);
  slv(ix, iy);
}
```

6.8. picks_theorem

```
struct Point{
    ll x,y;
    ll cross(const Point& o) const {return x*o.y-y*o
        .x;}
};
pair<ll,pair<ll,ll>> picks_theorem(vector<Point>&p){
    ll double_area=0, B=0;
    int n=p.size();
    for(int i=0,j=n-1;i<n;j=i++){
        double_area+=p[j].cross(p[i]);
        ll dx=abs(p[i].x-p[j].x);
        ll dy=abs(p[i].y-p[j].y);
        B+=gcd(dx,dy);
    }
    double_area=abs(double_area);
    ll I=(double_area-B+2)/2;
    //Real area = double_area/2.0
    return {double_area,{I,B}};
}
```

6.9. pointinpoligon

```
struct pt {
    long long x, y;
    pt() {}
    pt(long long _x, long long _y) : x(_x), y(_y) {}
    pt operator+(const pt &p) const { return pt(x +
        p.x, y + p.y); }
    pt operator-(const pt &p) const { return pt(x -
        p.x, y - p.y); }
    long long cross(const pt &p) const { return x *
        p.y - y * p.x; }
    long long dot(const pt &p) const { return x * p.
        x + y * p.y; }
    long long cross(const pt &a, const pt &b) const
        { return (a - *this).cross(b - *this); }
    long long dot(const pt &a, const pt &b) const {
        return (a - *this).dot(b - *this); }
```

```
    long long sqrLen() const { return this->dot(*
        this); }
};

bool lexComp(const pt &l, const pt &r) {
    return l.x < r.x || (l.x == r.x && l.y < r.y);
}

int sgn(long long val) { return val > 0 ? 1 : (val
    == 0 ? 0 : -1); }

vector<pt> seq;
pt translation;
int n;

bool pointInTriangle(pt a, pt b, pt c, pt point) {
    long long s1 = abs(a.cross(b, c));
    long long s2 = abs(point.cross(a, b)) + abs(
        point.cross(b, c)) + abs(point.cross(c, a));
    return s1 == s2;
}

void prepare(vector<pt> &points) {
    n = points.size();
    int pos = 0;
    for (int i = 1; i < n; i++) {
        if (lexComp(points[i], points[pos]))
            pos = i;
    }
    rotate(points.begin(), points.begin() + pos,
        points.end());

    n--;
    seq.resize(n);
    for (int i = 0; i < n; i++)
        seq[i] = points[i + 1] - points[0];
    translation = points[0];
}

bool pointInConvexPolygon(pt point) {
    point = point - translation;
    if (seq[0].cross(point) != 0 &&
            sgn(seq[0].cross(point)) != sgn(seq[0].
                cross(seq[n - 1])))
        return false;
    if (seq[n - 1].cross(point) != 0 &&
            sgn(seq[n - 1].cross(point)) != sgn(seq[
                n - 1].cross(seq[0])))
        return false;

    if (seq[0].cross(point) == 0)
        return seq[0].sqrLen() >= point.sqrLen();

    int l = 0, r = n - 1;
    while (r - l > 1) {
        int mid = (l + r) / 2;
        int pos = mid;
        if (seq[pos].cross(point) >= 0)
            l = mid;
        else
            r = mid;
    }
    int pos = l;
    return pointInTriangle(seq[pos], seq[pos + 1],
        pt(0, 0), point);
}
```

6.10. pointintpoligon

```
struct pt {
    long long x, y;
    pt() {}
    pt(long long _x, long long _y) : x(_x), y(_y) {}
    pt operator+(const pt &p) const { return pt(x +
        p.x, y + p.y); }
    pt operator-(const pt &p) const { return pt(x -
        p.x, y - p.y); }
    long long cross(const pt &p) const { return x *
        p.y - y * p.x; }
    long long dot(const pt &p) const { return x * p.
        x + y * p.y; }
    long long cross(const pt &a, const pt &b) const
        { return (a - *this).cross(b - *this); }
    long long dot(const pt &a, const pt &b) const {
        return (a - *this).dot(b - *this); }
    long long sqrLen() const { return this->dot(*
        this); }
```

```
};

bool lexComp(const pt &l, const pt &r) {
    return l.x < r.x || (l.x == r.x && l.y < r.y);
}

int sgn(long long val) { return val > 0 ? 1 : (val == 0 ? 0 : -1); }

vector<pt> seq;
pt translation;
int n;

bool pointInTriangle(pt a, pt b, pt c, pt point) {
    long long s1 = abs(a.cross(b, c));
    long long s2 = abs(point.cross(a, b)) + abs(point.cross(b, c)) + abs(point.cross(c, a));
    return s1 == s2;
}

void prepare(vector<pt> &points) {
    n = points.size();
    int pos = 0;
    for (int i = 1; i < n; i++) {
        if (lexComp(points[i], points[pos]))
            pos = i;
    }
    rotate(points.begin(), points.begin() + pos, points.end());

    n--;
    seq.resize(n);
    for (int i = 0; i < n; i++)
        seq[i] = points[i + 1] - points[0];
    translation = points[0];
}

bool pointInConvexPolygon(pt point) {
    point = point - translation;
    if (seq[0].cross(point) != 0 &&
            sgn(seq[0].cross(point)) != sgn(seq[0].cross(seq[n - 1])))
        return false;
    if (seq[n - 1].cross(point) != 0 &&
            sgn(seq[n - 1].cross(point)) != sgn(seq[n - 1].cross(seq[0])))
        return false;

    if (seq[0].cross(point) == 0)
        return seq[0].sqrLen() >= point.sqrLen();

    int l = 0, r = n - 1;
    while (r - l > 1) {
        int mid = (l + r) / 2;
        int pos = mid;
        if (seq[pos].cross(point) >= 0)
            l = mid;
        else
            r = mid;
    }
    int pos = l;
    return pointInTriangle(seq[pos], seq[pos + 1], pt(0, 0), point);
}
```

6.11. twoSegmentIntersec

```
struct pt {
    long long x, y;
    pt() {}
    pt(long long _x, long long _y) : x(_x), y(_y) {}
    pt operator-(const pt& p) const { return pt(x - p.x, y - p.y); }
    long long cross(const pt& p) const { return x * p.y - y * p.x; }
    long long cross(const pt& a, const pt& b) const { return (a - *this).cross(b - *this); }
};

int sgn(const long long& x) { return x >= 0 ? x ? 1 : 0 : -1; }

bool inter1(long long a, long long b, long long c, long long d) {
    if (a > b)
        swap(a, b);
    if (c > d)
```

```
        swap(c, d);
    return max(a, c) <= min(b, d);
}

bool check_inter(const pt& a, const pt& b, const pt&
    c, const pt& d) {
    if (c.cross(a, d) == 0 && c.cross(b, d) == 0)
        return inter1(a.x, b.x, c.x, d.x) && inter1(
            a.y, b.y, c.y, d.y);
    return sgn(a.cross(b, c)) != sgn(a.cross(b, d))
        &&
        sgn(c.cross(d, a)) != sgn(c.cross(d, b));
}
```

7. Primalidad

7.1. RabinMiller

```
#include <bits/stdc++.h>

using namespace std;


using u64 = uint64_t;
using u128 = __uint128_t;

u64 binpower(u64 base, u64 e, u64 mod) {
    u64 result = 1;
    base %= mod;
    while (e) {
        if (e & 1)
            result = (u128)result * base % mod;
        base = (u128)base * base % mod;
        e >>= 1;
    }
    return result;
}

bool check_composite(u64 n, u64 a, u64 d, int s) {
    u64 x = binpower(a, d, n);
    if (x == 1 || x == n - 1)
        return false;
    for (int r = 1; r < s; r++) {
        x = (u128)x * x % n;
        if (x == n - 1)
            return false;
    }
    return true;
};

bool MillerRabin(u64 n, int iter=5) { // returns
    true if n is probably prime, else returns false.
    cout<<iter<<endl;
    if (n < 4)
        return n == 2 || n == 3;

    int s = 0;
    u64 d = n - 1;
    while ((d & 1) == 0) {
        d >>= 1;
        s++;
    }

    for (int i = 0; i < iter; i++) {
        int a = 2 + rand() % (n - 3);
        if (check_composite(n, a, d, s))
            return false;
    }
    return true;
}


int main()
{

    cout<<MillerRabin(100000001,30)<<endl;
    return 0;
}
```

7.2. pollardRho

```
#include <iostream>
#include <cstdlib>
#include <cstdio>
#include <cmath>
#include <cassert>
```

```cpp
#include <map>

using namespace std;

typedef long long ll;

#define forn(i, n) for (int i = 0; i < (int)(n); i++)
#define forsn(i, s, n) for (int i = int(s); i < (int)(n); i++)

// rabin miller

ll potlog(ll a, ll b, const ll M)
{
    ll res = 1;
    while (b)
    {
        if (b % 2)
            res = (__int128(res) * a) % M;
        a = (__int128(a) * a) % M;
        b /= 2;
    }
    return res;
}

bool primo(ll n)
{
    if (n < 2)
        return false;
    if (n == 2)
        return true;
    ll D = n - 1, S = 0;
    while (D % 2 == 0)
    {
        D /= 2;
        S++;
    }
    // n-1 = 2^S * D
    static const int STEPS = 16;
    forn(pasos, STEPS)
    {
        const ll A = 1 + rand() % (n - 1);
        ll M = potlog(A, D, n);
        if (M == 1 || M == (n - 1))
            goto next;
        forn(k, S - 1)
        {
            M = (__int128(M) * M) % n;
            if (M == (n - 1))
                goto next;
        }
        return false;
    next:;
    }
    return true;
}

// pollard's rho

ll mcd(ll a, ll b) { return (a == 0) ? b : mcd(b % a, a); }

ll factor(ll n)
{
    static ll A, B;
    A = 1 + rand() % (n - 1);
    B = 1 + rand() % (n - 1);
#define f(x) ((__int128(x) * (x + B)) % n + A)
    ll x = 2, y = 2, d = 1;
    while (d == 1 || d == -1)
    {
        x = f(x);
        y = f(f(y));
        d = mcd(x - y, n);
    }
    return abs(d);
}

map<ll, ll> fact;

void factorize(ll n)
{
    assert(n > 0);
    while (n > 1 && !primo(n))
    {
        ll f;
        do
```

```
        {
            f = factor(n);
        } while (f == n);
        n /= f;
        factorize(f);
        for (auto &it : fact)
            while (n % it.first == 0)
            {
                n /= it.first;
                it.second++;
            }
    }
    if (n > 1)
        fact[n]++;
}

int main()
{
    ll N;
    while (cin >> N && N)
    {
        fact.clear();
        factorize(N);
        for (const auto &it : fact)
            cout << it.first << "^" << it.second <<
                " ";
        cout << endl;
    }
    return 0;
}
```

8. Graphs

8.1. 2-sat

```
struct TwoSatSolver {
    int n_vars;
    int n_vertices;
    vector<vector<int>> adj, adj_t;
    vector<bool> used;
    vector<int> order, comp;
    vector<bool> assignment;

    TwoSatSolver(int _n_vars) : n_vars(_n_vars),
        n_vertices(2 * n_vars), adj(n_vertices),
        adj_t(n_vertices), used(n_vertices), order(),
         comp(n_vertices, -1), assignment(n_vars) {
        order.reserve(n_vertices);
    }
    void dfs1(int v) {
        used[v] = true;
        for (int u : adj[v]) {
            if (!used[u])
                dfs1(u);
        }
        order.push_back(v);
    }

    void dfs2(int v, int cl) {
        comp[v] = cl;
        for (int u : adj_t[v]) {
            if (comp[u] == -1)
                dfs2(u, cl);
        }
    }

    bool solve_2SAT() {
        order.clear();
        used.assign(n_vertices, false);
        for (int i = 0; i < n_vertices; ++i) {
            if (!used[i])
                dfs1(i);
        }

        comp.assign(n_vertices, -1);
        for (int i = 0, j = 0; i < n_vertices; ++i)
            {
            int v = order[n_vertices - i - 1];
            if (comp[v] == -1)
                dfs2(v, j++);
        }

        assignment.assign(n_vars, false);
        for (int i = 0; i < n_vertices; i += 2) {
            if (comp[i] == comp[i + 1])
                return false;
            assignment[i / 2] = comp[i] > comp[i +
```

```
                1];
        }
        return true;
    }

    void add_disjunction(int a, bool na, int b, bool
        nb) {
        // na and nb signify whether a and b are to
           be negated
        a = 2 * a ^ na;
        b = 2 * b ^ nb;
        int neg_a = a ^ 1;
        int neg_b = b ^ 1;
        adj[neg_a].push_back(b);
        adj[neg_b].push_back(a);
        adj_t[b].push_back(neg_a);
        adj_t[a].push_back(neg_b);
    }

    static void example_usage() {
        TwoSatSolver solver(3); // a, b, c
        solver.add_disjunction(0, false, 1, true);
           //     a  v  not b
        solver.add_disjunction(0, true, 1, true);
           // not a  v  not b
        solver.add_disjunction(1, false, 2, false);
           //     b  v      c
        solver.add_disjunction(0, false, 0, false);
           //     a  v      a
        assert(solver.solve_2SAT() == true);
        auto expected = vector<bool>(True, False,
           True);
        assert(solver.assignment == expected);
    }
};
```

8.2. 2SAT

```
    #include <bits/stdc++.h>
    //#include <ext/pb_ds/assoc_container.hpp>
    //#include <ext/pb_ds/tree_policy.hpp>
    #define mp              make_pair
    #define pb              push_back
    #define all(a)          (a).begin(), (a).end()
    #define sz(a)           a.size()
    #define srt(a)          sort(all(a))
    #define mem(a, h)       memset(a, (h), sizeof(a)
        )
    #define f               first
    #define s               second
    #define forr(i, b, e)   for(int i = b; i < e; i
        ++)
    #define forg(i, b, e, m)    for(int i = b; i < e
        ; i+=m)
    //int in(){int r=0,c;for(c=getchar();c<=32;c=
        getchar());if(c=='-') return -in();for(;c>32;
        r=(r<<1)+(r<<3)+c-'0',c=getchar());return r;}

    using namespace std;
    //using namespace __gnu_pbds;

    typedef long long       ll;
    typedef unsigned long long      ull;
    typedef pair<int, int>  ii;
    typedef vector<int>     vi;
    typedef vector<ii>      vii;
    typedef vector<ll>      vll;
    //typedef tree<int,null_type,less<int>,
        rb_tree_tag,tree_order_statistics_node_update
        > ordered_set;
    //find_by_order kth largest  order_of_key <
    const int tam = 1200;
    const int MOD = 1e9+7;
    const int MOD1 = 988244535;
    const double EPS = 1e-9;
    const double PI = acos(-1);
    #define forite(it,a,b) for(auto it=a;it!=b;it++)
    vector<int> g[tam],ginv[tam];

    int n,m;
    int neg(int num)
    {
        if (num>n)
            return num-n;
        return num+n;
    }
```

```
    void add(int iz,int der)
    {
        g[iz].pb(der);
        g[neg(der)].pb(neg(iz));

        ginv[der].pb(iz);
        ginv[neg(iz)].pb(neg(der));
    }
    bool vis[tam],valor[tam];vector<int> orden;
    void dfs0(int num)
    {
        vis[num]=1;
        for(int hijo:g[num])
        {
            if (!vis[hijo])
                dfs0(hijo);
        }
        orden.pb(num);
    }
    int componente[tam],scc;
    void dfs1(int num)
    {
        vis[num]=1;
        componente[num]=scc;
        for(int hijo: ginv[num])
        {
            if (!vis[hijo])
            dfs1(hijo);
        }
    }
    int main()
    {
        ios::sync_with_stdio(0); cin.tie(0);

        cin>>n>>m;
        int iz,der,peso;
        forr(i,0,m)
        {
            cin>>iz>>der>>peso;
            if (peso==1)
            {
                add(iz,der);
                add(der,iz);
            }
            else
            {
                add(neg(iz),der);
                add(iz,neg(der));
            }
        }
        memset(vis,0,sizeof vis);
        forr(i,1,2*n+1)
        {
            if (!vis[i])
                dfs0(i);
        }
        memset(vis,0,sizeof vis);
        scc=0;

        forr(i,0,orden.size())
        {
            iz=orden[orden.size()-1-i];
            if (!vis[iz]){
                scc++;
                dfs1(iz);
            }
        }
        bool invalid=false;
        vector<int> respuesta;
        for(int i=1;i<=n;i++)
        {
            //cout<<componente[i]<<' '<<componente[i
               +n]<<endl;
            if (componente[i]==componente[i+n])
                invalid=1;
            if (componente[i]>componente[i+n])
            {
                respuesta.pb(i);
                valor[i]=1;
            }
            else valor[i]=0;
        }
        if (invalid==false)
        {
            cout<<respuesta.size()<<endl;
            for(int num:respuesta)
                cout<<num<<" ";
        }
```

```
        else
        {
            cout<<"Impossible\n";
        }
    }
    // read the question correctly (is y a vowel?
       what are the exact constraints?)
    // look out for SPECIAL CASES (n=1?) and
       overflow (ll vs int?) ARRAY OUT OF BOUNDSS2
```

8.3. Algoritmo_hungaro_de_asignacion

```
int n, m;
int a[n+1][m+1]; // matriz de costos, 1-index
vector<int> u(n+1), v(m+1), p(m+1), way(m+1);
for (int i = 1; i <= n; ++i) {
  p[0] = i;
  int j0 = 0;
  vector<int> minv(m + 1, INF);
  vector<char> used(m + 1, false);
  do {
    used[j0] = true;
    int i0 = p[j0], delta = INF, j1;
    for (int j = 1; j <= m; ++j)
      if ( !used[j] ) {
        int cur = a[i0][j] - u[i0] - v[j];
        if ( cur < minv[j] )
          minv[j] = cur, way[j] = j0;
        if ( minv[j] < delta )
          delta = minv[j], j1 = j;
      }
    for (int j = 0; j <= m; ++j)
      if ( used[j] )
        u[ p[j] ] += delta, v[j] -= delta;
      else
        minv[j] -= delta;
    j0 = j1;
  } while ( p[j0] != 0 );
  do {
    int j1 = way[j0];
    p[j0] = p[j1];
    j0 = j1;
  } while ( j0 ) ;
}

vector<int> ans(n+1); // Para cada fila, en que
   columna esta el resultado.
for (int j = 1; j <= m; ++j)
  ans[ p[j] ] = j;

int cost = -v[0];
```

8.4. FordFulkerson

```
//O(E*F)donde F flujo maximo

int dfs(int v, int t, int pushed) {
    if (v == t) return pushed;
    vis[v] = 1;

    for (auto &e : G[v]) {
        if (!vis[e.to] && e.cap > 0) {
            int f = dfs(e.to, t, min(pushed, e.cap))
               ;
            if (f) {
                e.cap -= f;
                G[e.to][e.rev].cap += f;
                return f;
            }
        }
    }
    return 0;
}

int maxflow(int s, int t) {
    int flow = 0;
    while (true) {
        fill(vis.begin(), vis.end(), 0);
        int pushed = dfs(s, t, INF);
        if (!pushed) break;
        flow += pushed;
    }
    return flow;
}
```

8.5. HoprofKarp

```

//complejidad Exraiz(v)
struct HopcroftKarp{
    const int INF = 1e9 + 7;
    HopcroftKarp(){}
    HopcroftKarp(int nLeft, int nRight){ init(nLeft,
        nRight); }

    void init(int nLeft, int nRight){
        L = nLeft, R = nRight;
        G.clear(), G.resize(L);
        pairU.assign(L, -1), pairV.assign(R, -1);
        dist.resize(L), Q.resize(L);
    }

    int L, R;
    vector<vi> G;
    vi pairU, pairV, dist, Q;

    void addEdge(int u, int v){ G[u].pb(v); }

    bool bfs(){
        int qh = 0, qt = 0;
        for(int u = 0; u < L; u++){
            if(pairU[u] == -1) dist[u] = 0, Q[qt++]
                = u;
            else dist[u] = INF;
        }
        bool ok = 0;
        while(qh < qt){
            int u = Q[qh++];
            for(int v : G[u]){
                int u2 = pairV[v];
                if(u2 == -1) ok = 1;
                else if(dist[u2] == INF) dist[u2] =
                    dist[u] + 1, Q[qt++] = u2;
            }
        }
        return ok;
    }

    bool dfs(int u){
        for(int v : G[u]){
            int u2 = pairV[v];
            if(u2 == -1 || (dist[u2] == dist[u] + 1
                && dfs(u2))){
                pairU[u] = v, pairV[v] = u;
                return 1;
            }
        }
        dist[u] = INF;
        return 0;
    }

    int maxMatching(){
        pairU.assign(L, -1), pairV.assign(R, -1);
        int match = 0;
        while(bfs()) for(int u = 0; u < L; u++)
            if(pairU[u] == -1 && dfs(u)) match++;
        return match;
    }
};
```

8.6. MinCost-MaxFlow_DEsopo-Pape

```
struct Edge
{
    int from, to, capacity, cost;
};

vector<vector<int>> adj, cost, capacity;

const int INF = 1e9;

void shortest_paths(int n, int v0, vector<int>& d,
    vector<int>& p) {
    d.assign(n, INF);
    d[v0] = 0;
    vector<int> m(n, 2);
    deque<int> q;
    q.push_back(v0);
    p.assign(n, -1);

    while (!q.empty()) {
        int u = q.front();
```

```
        q.pop_front();
        m[u] = 0;
        for (int v : adj[u]) {
            if (capacity[u][v] > 0 && d[v] > d[u] +
                cost[u][v]) {
                d[v] = d[u] + cost[u][v];
                p[v] = u;
                if (m[v] == 2) {
                    m[v] = 1;
                    q.push_back(v);
                } else if (m[v] == 0) {
                    m[v] = 1;
                    q.push_front(v);
                }
            }
        }
    }
}

int min_cost_flow(int N, vector<Edge> edges, int K,
    int s, int t) {
    adj.assign(N, vector<int>());
    cost.assign(N, vector<int>(N, 0));
    capacity.assign(N, vector<int>(N, 0));
    for (Edge e : edges) {
        adj[e.from].push_back(e.to);
        adj[e.to].push_back(e.from);
        cost[e.from][e.to] = e.cost;
        cost[e.to][e.from] = -e.cost;
        capacity[e.from][e.to] = e.capacity;
    }

    int flow = 0;
    int cost = 0;
    vector<int> d, p;
    while (flow < K) {
        shortest_paths(N, s, d, p);
        if (d[t] == INF)
            break;

        // find max flow on that path
        int f = K - flow;
        int cur = t;
        while (cur != s) {
            f = min(f, capacity[p[cur]][cur]);
            cur = p[cur];
        }

        // apply flow
        flow += f;
        cost += f * d[t];
        cur = t;
        while (cur != s) {
            capacity[p[cur]][cur] -= f;
            capacity[cur][p[cur]] += f;
            cur = p[cur];
        }
    }

    if (flow < K)
        return -1;
    else
        return cost;
}
```

8.7. MinCostMaxFlow_Dijkstra

```
// O(min(E^2*V^2, E*V*FLOW))

#include <bits/stdc++.h>
//#include <ext/pb_ds/assoc_container.hpp>
//#include <ext/pb_ds/tree_policy.hpp>
typedef long long       ll;
#define pb                emplace_back
#define mp                make_pair
#define all(a)            (a).begin(), (a).end()
#define clr(a,h)          memset(a, (h), sizeof(a))
#define F first
#define S second
#define fore(i, b, e)   for (int i = (int)b; i < (
    int)e; i++)
int faster_in(){int r=0,c;for(c=getchar();c<=32;c=
    getchar());if(c=='-') return -faster_in();for(;c
    >32;r=(r<<1)+(r<<3)+c-'0',c=getchar());return r;}

using namespace std;
//using namespace __gnu_pbds;
```

```

typedef pair<int, int>  ii;
typedef vector<int>     vi;
typedef vector<ii>      vii;
typedef vector<ll>      vll;
//typedef tree<int,null_type,less<int>,rb_tree_tag,
    tree_order_statistics_node_update> ordered_set;
const int INF = int(1e9 + 7);
const double EPS = 1e-9;
const double PI = acos(-1);

const int tam = 110;

struct Edge
{
  int to, rev;
  int flow, cap;
  int cost;
};
vector<Edge> G[tam];

void addEdge(int s, int t, int cap, int cost)
{
  Edge a = {t, (int)G[t].size(), 0, cap, cost};
  Edge b = {s, (int)G[s].size(), 0, 0, -cost};
  G[s].pb(a);
  G[t].pb(b);
}

int n, m; // En vez de n es mejor usar una variable
    diferente como 'nodes'
          // La cantidad de nodos en la red de flujo
              suele ser difente a la
          // cantidad en el grafo original

int S, T;
vector<int> dist;
int pot[tam], curFlow[tam], prevNode[tam], prevEdge[
    tam];
int Q[tam];
bool inQueue[tam];

void bellmanFord()
{
  fill(pot, pot + n, INF);
  pot[S] = 0;
  int qt = 0;
  Q[qt++] = S;
  for (int qh = 0; (qh - qt) % n != 0; qh++)
  {
    int u = Q[qh % n];
    inQueue[u] = false;
    for (int i = 0; i < (int)G[u].size(); i++)
    {
      Edge &e = G[u][i];
      if (e.cap <= e.flow) continue;
      int v = e.to;
      int newDist = pot[u] + e.cost;
      if (pot[v] > newDist)
      {
        pot[v] = newDist;
        if (!inQueue[v])
        {
          Q[qt++ % n] = v;
          inQueue[v] = true;
        }
      }
    }
  }
}

ii MinCostMaxFlow()
{
  bellmanFord();
  int flow = 0;
  int flowCost = 0;
  while (true) // always a good start for an
      algorithm :v
  {
    set<ii> s;
    s.insert( mp(0, S) );
    dist.assign(n, INF);
    dist[S] = 0;
    curFlow[S] = INF;
    while (s.size() > 0)
    {
      int u = s.begin() -> second;
      int actDist = s.begin() -> first;
```

```
      s.erase(s.begin());
      if (actDist > dist[u]) continue;
      for (int i = 0; i < (int)G[u].size(); i++)
      {
        Edge &e = G[u][i];
        int v = e.to;
        if (e.cap <= e.flow) continue;
        int newDist = actDist + e.cost + pot[u] -
           pot[v];
        if (newDist < dist[v])
        {
          dist[v] = newDist;
          s.insert( mp(newDist, v) );
          prevNode[v] = u;
          prevEdge[v] = i;
          curFlow[v] = min(curFlow[u], e.cap - e.
             flow);
        }
      }
    }
    if (dist[T] == INF)
      break;
    for (int i = 0; i < n; i++)
      pot[i] += dist[i];
    int df = curFlow[T];
    flow += df;
    for (int v = T; v != S; v = prevNode[v])
    {
      Edge &e = G[prevNode[v]][prevEdge[v]];
      e.flow += df;
      G[v][e.rev].flow -= df;
      flowCost += df * e.cost;
    }
  }
  return mp(flow, flowCost);
}

int main()
{
    std::ios::sync_with_stdio(false); cin.tie(0);
    //freopen("","r",stdin);
    //freopen("","w",stdout);
    cin >> n >> m;
    S = 0, T = n-1;
    fore(i, 0, m)
    {
      int a, b, cap, cost;
      cin >> a >> b >> cap >> cost;
      a--; b--;
      addEdge(a, b, cap, cost);
    }
    cout << MinCostMaxFlow().S << '\n';
    return 0;
}
// PLUS ULTRA!
```

8.8. articulationpoint

```
int n; // number of nodes
vector<vector<int>> gf; // gfacency list of graph

vector<bool> vis;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
    vis[v] = true;
    tin[v] = low[v] = timer++;
    int children=0;
    for (int to : gf[v]) {
        if (to == p) continue;
        if (vis[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] >= tin[v] && p!=-1)
                IS_CUTPOINT(v);
            ++children;
        }
    }
    if(p == -1 && children > 1)
        IS_CUTPOINT(v);
}

void find_cutpoints() {
    timer = 0;
```

```
    vis.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);
    for (int i = 0; i < n; ++i) {
        if (!vis[i])
            dfs (i);
    }
}
```

8.9. bridges

```
void IS_BRIDGE(int v,int to); // some function to
   process the found bridge
int n; // number of nodes
vector<vector<int>> gf; // gfacency list of graph

vector<bool> vis;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
    vis[v] = true;
    tin[v] = low[v] = timer++;
    bool parent_skipped = false;
    for (int to : gf[v]) {
        if (to == p && !parent_skipped) {
            parent_skipped = true;
            continue;
        }
        if (vis[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] > tin[v])
                IS_BRIDGE(v, to);
        }
    }
}

void find_bridges() {
    timer = 0;
    vis.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);
    for (int i = 0; i < n; ++i) {
        if (!vis[i])
            dfs(i);
    }
}
```

8.10. centroid

```
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;
const int MAXN=1e5;
vector<int> gf[MAXN];
bool die[MAXN]={};
int ch[MAXN]={};
int n,q;

vector<int> min_dist(MAXN,1e9);

int get_sz_subTree(int nodo,int p)
{
    ch[nodo]=1;
    for(int &h:gf[nodo])
    {
        if(die[h] or h==p)continue;
        ch[nodo]+=get_sz_subTree(h,nodo);
    }
    return ch[nodo];
}

int get_centroid(int nodo,int p,int sz)
{
    for(int &h:gf[nodo])
    {
        if(die[h] or h==p)continue;
        if(ch[h]*2>sz)return get_centroid(h,nodo,sz)
            ;
    }
    return nodo;
}
```

```cpp

vector<pair<int,int>> ancenstros[MAXN];

void get_dist(int nodo,int p,int lvl,int cent)
{
    ancenstros[nodo].push_back({cent,lvl});
    for(int &h:gf[nodo])
    {
        if(h!=p and !die[h])
        {
            get_dist(h,nodo,lvl+1,cent);
        }
    }
}

void sexo_decomposition(int nodo)
{
    int cent=get_centroid(nodo,-1,get_sz_subTree(
       nodo,-1));
    //algo

    for(int &h:gf[cent])
    {
        if(!die[h])
        {
            get_dist(h,cent,1,cent);
        }
    }


    die[cent]=1;
    for(int &h:gf[cent])if(!die[h])
       sexo_decomposition(h);
}

void pintar(int nodo)
{
    for(auto &[ancestro,dis]:ancenstros[nodo])
    {
        min_dist[ancestro]=min(min_dist[ancestro],
           dis);
    }
    min_dist[nodo]=0;
}


int query(int nodo)
{
    int res=min_dist[nodo];

    for(auto [ancient,dist]:ancenstros[nodo])
    {
        res=min(res,dist+min_dist[ancient]);
    }
    return res;

}


int main()
{
    cin.tie(nullptr)->sync_with_stdio(false);
    cin>>n>>q;

    for(int i=0;i<n-1;i++)
    {
        int a,b;cin>>a>>b;
        a--;b--;
        gf[a].push_back(b);
        gf[b].push_back(a);
    }

    sexo_decomposition(0);

    pintar(0);

    while(q--)
    {
        int type;cin>>type;
        int v;cin>>v;v--;
        if(type&1)
        {
            pintar(v);
        }
        else
        {
            cout<<query(v)<<"\n";
        }
    }
```

```
118
119
120
121
122     return 0;
123
124 }
```

8.11. componentes_fuertemente_conexos

```
1  #include <bits/stdc++.h>
2
3  const int N = 1e5 + 1;
4
5  // adj_t is the transpose of adj
6  std::vector<int> adj[N], adj_t[N];
7  std::vector<int> order;
8  std::vector<int> vis(N), id(N);
9
10 // calculates the order in which nodes are processed
11 void dfs1(int v) {
12   vis[v] = true;
13   for (int u : adj[v]) {
14     if (!vis[u]) { dfs1(u); }
15   }
16   order.push_back(v);
17 }
18
19 /*
20  * traverse one SCC at a time and
21  * assign component ID to all nodes in one SCC
22  */
23 void dfs2(int x, int comp) {
24   vis[x] = true;
25
26   for (int u : adj_t[x]) {
27     if (!vis[u]) { dfs2(u, comp); }
28   }
29   id[x] = comp;
30 }
31
32 int main() {
33   int n, m;
34   std::cin >> n >> m;
35
36   for (int i = 1; i <= m; i++) {
37     int u, v;
38     std::cin >> u >> v;
39     adj[u].push_back(v);
40     adj_t[v].push_back(u);
41   }
42
43   vis.assign(N, false);
44
45   for (int i = 1; i <= n; i++) {
46     if (!vis[i]) { dfs1(i); }
47   }
48
49   vis.assign(N, false);
50   std::reverse(begin(order), end(order));
51   int comps = 0;
52   for (int v : order) {
53     if (!vis[v]) {
54       comps++;  // counter for SCC
55       dfs2(v, comps);
56     }
57   }
58
59   std::cout << comps << '\n';
60
61   for (int i = 1; i <= n; i++) { std::cout << id[i]
       << " \n"[i == n]; }
62 }
```

8.12. dijstra

```
1  void dijstra(int s)
2  {
3      set<pair<int,int>> pq;
4      dis.assign(n,INF);
5      dis[s]=0;
6      pq.insert({0,s});
7      while(!pq.empty())
8      {
9          pair<int,int> top=*pq.begin();
10         pq.erase(pq.begin());
```

```
        int v=top.second;
        int d=top.first;

        for(auto [v2,cost]:gf[v])
        {
            if(dis[v2]>dis[v]+cost)
            {
                if(dis[v2]<INF)
                {
                    pq.erase(pq.find(make_pair(dis[
                        v2],v2)));
                }
                dis[v2]=dis[v]+cost;
                pq.insert(make_pair(dis[v2],v2));
            }
        }
    }
}
```

8.13. dinic

```
struct FlowEdge {
    int v, u;
    long long cap, flow = 0;
    FlowEdge(int v, int u, long long cap) : v(v), u(
        u), cap(cap) {}
};

struct Dinic {
    const long long flow_inf = 1e18;
    vector<FlowEdge> edges;
    vector<vector<int>> gf;
    int n, m = 0;
    int s, t;
    vector<int> level, ptr;
    queue<int> q;

    Dinic(int n, int s, int t) : n(n), s(s), t(t) {
        gf.resize(n);
        level.resize(n);
        ptr.resize(n);
    }

    void add_edge(int v, int u, long long cap) {
        edges.emplace_back(v, u, cap);
        edges.emplace_back(u, v, 0);
        gf[v].push_back(m);
        gf[u].push_back(m + 1);
        m += 2;
    }

    bool bfs() {
        while (!q.empty()) {
            int v = q.front();
            q.pop();
            for (int id : gf[v]) {
                if (edges[id].cap == edges[id].flow)
                    continue;
                if (level[edges[id].u] != -1)
                    continue;
                level[edges[id].u] = level[v] + 1;
                q.push(edges[id].u);
            }
        }
        return level[t] != -1;
    }

    long long dfs(int v, long long pushed) {
        if (pushed == 0)
            return 0;
        if (v == t)
            return pushed;
        for (int& cid = ptr[v]; cid < (int)gf[v].
            size(); cid++) {
            int id = gf[v][cid];
            int u = edges[id].u;
            if (level[v] + 1 != level[u])
                continue;
            long long tr = dfs(u, min(pushed, edges[
                id].cap - edges[id].flow));
            if (tr == 0)
                continue;
            edges[id].flow += tr;
            edges[id ^ 1].flow -= tr;
            return tr;
        }
```

```
        return 0;
    }

    long long flow() {
        long long f = 0;
        while (true) {
            fill(level.begin(), level.end(), -1);
            level[s] = 0;
            q.push(s);
            if (!bfs())
                break;
            fill(ptr.begin(), ptr.end(), 0);
            while (long long pushed = dfs(s,
                flow_inf)) {
                f += pushed;
            }
        }
        return f;
    }
};
```

8.14. dinitz

```
//O((V^2)*E)

//Dinic igual complejidad

struct Dinitz{
    const int INF = 1e9 + 7;
    Dinitz(){}
    Dinitz(int n, int s, int t) {init(n, s, t);}

    void init(int n, int s, int t)
    {
        S = s, T = t;
        nodes = n;
        G.clear(), G.resize(n);
        Q.resize(n);
    }
    struct flowEdge
    {
        int to, rev, f, cap;
    };

    vector<vector<flowEdge> > G;

    // Aniade arista (st -> en) con su capacidad
    void addEdge(int st, int en, int cap) {
        flowEdge A = {en, (int)G[en].size(), 0, cap
            };
        flowEdge B = {st, (int)G[st].size(), 0, 0};
        G[st].pb(A);
        G[en].pb(B);
    }

    int nodes, S, T; // asignar estos valores al
        armar el grafo G
                    // nodes = nodos en red de flujo
                        . Hacer G.clear(); G.resize(
                        nodes);
    vi work, lvl;
    vi Q;

    bool bfs() {
        int qt = 0;
        Q[qt++] = S;
        lvl.assign(nodes, -1);
        lvl[S] = 0;
        for (int qh = 0; qh < qt; qh++) {
            int v = Q[qh];
            for (flowEdge &e : G[v]) {
                int u = e.to;
                if (e.cap <= e.f || lvl[u] != -1)
                    continue;
                lvl[u] = lvl[v] + 1;
                Q[qt++] = u;
            }
        }
        return lvl[T] != -1;
    }

    int dfs(int v, int f) {
        if (v == T || f == 0) return f;
        for (int &i = work[v]; i < G[v].size(); i++)
            {
            flowEdge &e = G[v][i];
            int u = e.to;
```

```
            if (e.cap <= e.f || lvl[u] != lvl[v] +
                1) continue;
            int df = dfs(u, min(f, e.cap - e.f));
            if (df) {
                e.f += df;
                G[u][e.rev].f -= df;
                return df;
            }
        }
        return 0;
    }

    int maxFlow() {
        int flow = 0;
        while (bfs()) {
            work.assign(nodes, 0);
            while (true) {
                int df = dfs(S, INF);
                if (df == 0) break;
                flow += df;
            }
        }
        return flow;
    }
};
```

8.15. edmonkarp

```
//O(V*(E^2))

int n;
vector<vector<int>> capacity;
vector<vector<int>> adj;

int bfs(int s, int t, vector<int>& parent) {
    fill(parent.begin(), parent.end(), -1);
    parent[s] = -2;
    queue<pair<int, int>> q;
    q.push({s, INF});

    while (!q.empty()) {
        int cur = q.front().first;
        int flow = q.front().second;
        q.pop();

        for (int next : adj[cur]) {
            if (parent[next] == -1 && capacity[cur][
                next]) {
                parent[next] = cur;
                int new_flow = min(flow, capacity[
                    cur][next]);
                if (next == t)
                    return new_flow;
                q.push({next, new_flow});
            }
        }
    }

    return 0;
}

int maxflow(int s, int t) {
    int flow = 0;
    vector<int> parent(n);
    int new_flow;

    while (new_flow = bfs(s, t, parent)) {
        flow += new_flow;
        int cur = t;
        while (cur != s) {
            int prev = parent[cur];
            capacity[prev][cur] -= new_flow;
            capacity[cur][prev] += new_flow;
            cur = prev;
        }
    }

    return flow;
}
```

8.16. eulerianpath

```
int main() {
    int n;
    vector<vector<int>> g(n, vector<int>(n));
    // reading the graph in the adjacency matrix
```

```

    vector<int> deg(n);
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j)
            deg[i] += g[i][j];
    }

    int first = 0;
    while (first < n && !deg[first])
        ++first;
    if (first == n) {
        cout << -1;
        return 0;
    }

    int v1 = -1, v2 = -1;
    bool bad = false;
    for (int i = 0; i < n; ++i) {
        if (deg[i] & 1) {
            if (v1 == -1)
                v1 = i;
            else if (v2 == -1)
                v2 = i;
            else
                bad = true;
        }
    }

    if (v1 != -1)
        ++g[v1][v2], ++g[v2][v1];

    stack<int> st;
    st.push(first);
    vector<int> res;
    while (!st.empty()) {
        int v = st.top();
        int i;
        for (i = 0; i < n; ++i)
            if (g[v][i])
                break;
        if (i == n) {
            res.push_back(v);
            st.pop();
        } else {
            --g[v][i];
            --g[i][v];
            st.push(i);
        }
    }

    if (v1 != -1) {
        for (size_t i = 0; i + 1 < res.size(); ++i)
            {
            if ((res[i] == v1 && res[i + 1] == v2)
                ||
                (res[i] == v2 && res[i + 1] == v1))
                    {
                vector<int> res2;
                for (size_t j = i + 1; j < res.size
                    (); ++j)
                    res2.push_back(res[j]);
                for (size_t j = 1; j <= i; ++j)
                    res2.push_back(res[j]);
                res = res2;
                break;
            }
        }
    }

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            if (g[i][j])
                bad = true;
        }
    }

    if (bad) {
        cout << -1;
    } else {
        for (int x : res)
            cout << x << " ";
    }
}
```

8.17. floyd_warshall

```
for (int k = 0; k < n; ++k) {
```

```
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            if (d[i][k] < INF && d[k][j] < INF)
                d[i][j] = min(d[i][j], d[i][k] + d[k
                    ][j]);
        }
    }
}
```

8.18. hld

```

const int MAXN=2e5;
vector<int> gf[MAXN];
int parent[MAXN],depth[MAXN],heavy[MAXN],head[MAXN],
    pos[MAXN];
int cur_pos;
const ll MOD=1000000007;

int n;

int dfs(int nodo)
{
    int size=1;
    int max_c_size=0;
    for(int &c:gf[nodo])
    {
        if(c!=parent[nodo])
        {
            parent[c]=nodo;
            depth[c]=depth[nodo]+1;

            int c_size=dfs(c);
            size+=c_size;

            if(c_size>max_c_size)
            {
                max_c_size=c_size;
                heavy[nodo]=c;
            }
        }
    }
    return size;
}

void update_on_vertex(int a, int b, ll val)
{
    for(; head[a] != head[b]; b = parent[head[b]])
    {
        if(depth[head[a]] > depth[head[b]])
            swap(a,b);

        st.update(1,0,st.sz-1,pos[head[b]],pos[b],
            val);
    }

    if(depth[a] > depth[b])
        swap(a,b);

    st.update(1,0,st.sz-1,pos[a],pos[b],val);
}

ll query_on_vertex(int a, int b)
{
    ll res = 0;

    for(; head[a] != head[b]; b = parent[head[b]])
    {
        if(depth[head[a]] > depth[head[b]])
            swap(a,b);

        res = max(res , st.query(1,0,st.sz-1,pos[
            head[b]],pos[b]));
    }

    if(depth[a] > depth[b])
        swap(a,b);

    res = max(res , st.query(1,0,st.sz-1,pos[a],pos[
        b]));
    return res;
}


void decompose(int v,int h)
{
    head[v]=h;
```

```
    pos[v]=cur_pos++;
    if(heavy[v]!=-1)
    {
        decompose(heavy[v],h);
    }
    for(int &c:gf[v])
    {
        if(c!=parent[v] && c!=heavy[v])
        {
            decompose(c,c);
        }
    }
}

void init()
{
    // parent=vector<int>(n);
    // depth=vector<int>(n);
    // heavy=vector<int>(n);
    // head=vector<int>(n);
    // pos=vector<int>(n);
    for(int i=0;i<MAXN;i++)heavy[i]=-1;
    cur_pos=0;

    dfs(0);
    decompose(0,0);
}


void update(int a,int b, ll val)//es query y update
   a la vez
{
    for(;head[a]!=head[b];b=parent[head[b]])
    {
        if(depth[head[a]]>depth[head[b]])swap(a,b);
        // ll resdomer=st.query()
        st.update(1,0,st.sz-1,pos[head[b]],pos[b],
            val);
        // res=(res*resdomer)%MOD;
    }

    if(depth[a]>depth[b])swap(a,b);

    if(a != b)st.update(1,0,st.sz-1,pos[a]+1,pos[b],
        val);
}


ll query(int a,int b)//quety on edges
{
    ll res=0;

    for(;head[a]!=head[b];b=parent[head[b]])
    {
        if(depth[head[a]]>depth[head[b]])swap(a,b);
        // ll resdomer=st.query()
        ll redomer=st.query(1,0,st.sz-1,pos[head[b
            ]],pos[b]);
        res=(res+redomer);
    }

    if(depth[a]>depth[b])swap(a,b);


    ll last_heavy_path_res=0;
    if(a != b)last_heavy_path_res=st.query(1,0,st.sz
       -1,pos[a]+1,pos[b]);

    // ll last_heavy_path_res=1;
    res=(res+last_heavy_path_res);
    return res;
}
```

8.19. isDag

```
vector<vector<int>> gf;  // lista de adyacencia
vector<int> visited;     // 0 = no visitado, 1 =
   visitando, 2 = visitado

bool dfs(int u) {
    visited[u] = 1; // visitando
    for (int v : gf[u]) {
        if (visited[v] == 1) return true;  // ciclo
           detectado
        if (visited[v] == 0 && dfs(v)) return true;
    }
    visited[u] = 2; // visitado
```

```
    return false;
}

bool isDAG(int n) {
    visited.assign(n, 0);
    for (int i = 0; i < n; i++)
        if (visited[i] == 0 && dfs(i))
            return false;  // hay ciclo
    return true; // no hay ciclos
}
```

8.20. isDagv2

```
vector<vector<int>> gf;
vector<bool> vis;
set<int> st;
bool sw = 1;
int n;

void isDAG(int nodo)
{
    // no olvidar recorrer con un for todo
    //ya que no siempre estan conectados
    if (!sw)return;
    vis[nodo] = 1;
    st.insert(nodo);
    for (auto hijo : gf[nodo])
    {
        if (st.count(hijo) == 1)
        {
            sw = 0;
            return;
        }
        if (!vis[hijo])
        {
            vis[hijo] = 1;
            isDAG(hijo);
        }
    }
    st.erase(nodo);
}
```

8.21. khun

```
//O(LxE)

struct Kuhn{
    Kuhn(){}
    Kuhn(int nLeft, int nRight){ init(nLeft, nRight)
        ; }

    void init(int nLeft, int nRight){
        L = nLeft, R = nRight;
        G.clear(), G.resize(L);
        mt.assign(R, -1);
        used.assign(L, 0);
        cc = 1;
    }

    int L, R, cc;
    vector<vi> G;
    vi mt, used; // mt[to] = nodo izquierdo
        emparejado con to

    void addEdge(int u, int v){ G[u].pb(v); }

    bool dfs(int v){
        if(used[v] == cc) return 0;
        used[v] = cc;
        for(int to : G[v]) if(mt[to] == -1 || dfs(mt
            [to])) return mt[to] = v, 1;
        return 0;
    }

    int maxMatching(){
        mt.assign(R, -1);
        int match = 0;
        for(int v = 0; v < L; v++){
            ++cc;
            if(dfs(v)) match++;
        }
        return match;
    }
};
```

8.22. khunMaxbipartiteMatching

```cpp
//aca a la hora de aniadir un edge, debemos ya tener que a es el grupo 1 y b el grupo 2

struct Kuhn {
    int n, m;
    vector<vector<int>> adj;
    vector<int> mt, vis;

    Kuhn(int n, int m) : n(n), m(m) {
        adj.resize(n);
        mt.assign(m, -1);
        vis.assign(n, 0);
    }

    void addEdge(int u, int v) {
        adj[u].push_back(v);
    }

    bool dfs(int u) {
        if (vis[u]) return false;
        vis[u] = 1;

        for (int v : adj[u]) {
            if (mt[v] == -1 || dfs(mt[v])) {
                mt[v] = u;
                return true;
            }
        }

        return false;
    }

    int maximum_matching() {
        int ans = 0;

        for (int u = 0; u < n; ++u) {
            fill(vis.begin(), vis.end(), 0);
            ans += dfs(u);
        }

        return ans;
    }
};
```

8.23. lcabinaryliftin

```cpp
int n, l;
vector<vector<int>> gf;

int timer;
vector<int> tin, tout;
vector<vector<int>> up;

void dfs(int v, int p)
{
    tin[v] = ++timer;
    up[v][0] = p;
    for (int i = 1; i <= l; ++i)
        up[v][i] = up[up[v][i-1]][i-1];

    for (int u : gf[v]) {
        if (u != p)
            dfs(u, v);
    }

    tout[v] = ++timer;
}

bool is_ancestor(int u, int v)
{
    return tin[u] <= tin[v] && tout[u] >= tout[v];
}

int lca(int u, int v)
{
    if (is_ancestor(u, v))
        return u;
    if (is_ancestor(v, u))
        return v;
    for (int i = l; i >= 0; --i) {
        if (!is_ancestor(up[u][i], v))
            u = up[u][i];
    }
    return up[u][0];
}
```

```
40
41 void preprocess(int root) {
42     tin.resize(n);
43     tout.resize(n);
44     timer = 0;
45     l = ceil(log2(n));
46     up.assign(n, vector<int>(l + 1));
47     dfs(root, root);
48 }
```

8.24. toposort

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  int n; // number of vertices
4  vector<vector<int>> gf;
5  vector<bool> vis;
6  vector<int> ans;
7
8  void dfs(int v) {//0i
9      vis[v] = true;
10     for (int u : gf[v]) {
11         if (!vis[u]) {
12             dfs(u);
13         }
14     }
15     ans.push_back(v);
16 }
17
18 void topological_sort() {
19     vis.assign(n, false);
20     ans.clear();
21     for (int i = 0; i < n; ++i) {
22         if (!vis[i]) {
23             dfs(i);
24         }
25     }
26     reverse(ans.begin(), ans.end());
27 }
```

9. Overloads

9.1. pq

```
1  struct cmp
2  {
3      //mayor tiene prioridad
4      bool operator()(const int& a, const int& b)
           const {
5          return a<b;
6      }
7  };
8
9  //priority_queue<Data,vector<Data>,cmp> pq
```

9.2. set

```
1  struct cmpST
2  {
3      //de menor a mayor
4      bool operator()(const int &a,const int &b)
5      {
6          return a<b;
7      }
8  };
9
10 //set<int,cmpST> st;
```

10. Dp

10.1. LCA_DP

```
1  #include <bits/stdc++.h>
2  #define clr(a,h)           memset(a, (h), sizeof(a))
3
4  using namespace std;
5
6  const int tam = 1010; // Max Nodos
7  const int Log2Tam = (log(tam)/log(2)) + 3;
8
9  vector< vi > g;
10 int dp[tam][Log2Tam], depth[tam];
```

```cpp
int n, m, root; // nodos, aristas, raiz del arbol

void initDFS(int v, int p, int d)
{
  dp[v][0] = p; // padre inmediato
  depth[v] = d;
  for (int u : g[v])
  {
    if (u == p) continue;
    initDFS(u, v, d+1);
  }
}

void initLCA()
{
  clr(dp, -1);
  initDFS(root, -1, 0);
  for (int pot = 1; pot < Log2Tam; pot++)
  {
    for (int v = 0; v < n; v++)
    {
      if (dp[v][pot-1] == -1) continue;
      dp[v][pot] = dp[ dp[v][pot-1] ][pot-1];
    }
  }
}

int LCA(int a, int b)
{
  // b siempre debajo o al mismo nivel que a
  if (depth[a] > depth[b]) swap(a, b);

  int diff = depth[b] - depth[a];
  for (int pot = Log2Tam-1; pot >= 0; pot--)
  {
    if ( ( diff & (1 << pot) ) )
    {
      b = dp[b][pot];
    }
  }
  if (a == b) return a;
  for (int pot = Log2Tam-1; pot >= 0; pot--)
  {
    if (dp[a][pot] != dp[b][pot])
    {
      a = dp[a][pot];
      b = dp[b][pot];
    }
  }
  int lca = dp[a][0];
  return lca;
}

int main()
{
    /*
        Iniciar el arbol en el grafo g
        Asignar la raiz en la variable root

        Llamar initLCA() luego de crear el arbol
        LCA(a, b) devuelve el LCA de los nodos a y b
    */
    return 0;
}
// PLUS ULTRA!
```

10.2. covexHullTrick

```cpp
struct ConvexHull
{
    struct Ln
    {
        ll m,b;//m pendiente m1>m2>m3... b constante
    };

    vector<Ln> cht;
    bool tapa(Ln l1, Ln l2, Ln l3)//verifica si con
       al agregar la linea 3 la 2 se vuelve
       irrelevante
    {
        return (__int128)(l3.m-l2.m)*(l1.b-l3.b)>=(
           __int128)(l2.b-l3.b)*(l3.m-l1.m);
        //<= para maximos
    }
    void addLine(Ln nueva)
    {
        int sz=cht.size();
```

```
        while(cht.size()>1)
        {
            sz=cht.size();
            if(tapa(cht[sz-2],cht[sz-1],nueva))cht.
               pop_back();
            else break;
        }
        cht.push_back(nueva);
    }

    ll f(int p,ll x)
    {
        return cht[p].m*x+cht[p].b;
    }

    int query(ll x)//te da el indice de la mejor
       linea
    {
        int l = 0, r = cht.size() - 1;

        auto f = [&](int i)
        {
            return cht[i].m * x + cht[i].b;
        };

        while(l < r)
        {
            int mid = (l + r) / 2;
            //>= para maximos
            if(f(mid) <= f(mid + 1))r = mid;
            else l = mid + 1;
        }
        return l;
    }
};
```

10.3. lis

```
// Longest Increasing Subsequence - O(nlogn)
//
// dp(i) = max j<i { dp(j) | a[j] < a[i] } + 1
//

// int dp[N], v[N], n, lis;

memset(dp, 63, sizeof dp);
for (int i = 0; i < n; ++i) {
  // increasing: lower_bound
  // non-decreasing: upper_bound
  int j = lower_bound(dp, dp + lis, v[i]) - dp;
  dp[j] = min(dp[j], v[i]);
  lis = max(lis, j + 1);
}
```

10.4. list

```
int lis(vector<int> const& a) {
    int n = a.size();
    const int INF = 1e9;
    vector<int> d(n+1, INF);
    d[0] = -INF;

    for (int i = 0; i < n; i++) {
        int l = upper_bound(d.begin(), d.end(), a[i
           ]) - d.begin();
        if (d[l-1] < a[i] && a[i] < d[l])
            d[l] = a[i];
    }

    int ans = 0;
    for (int l = 0; l <= n; l++) {
        if (d[l] < INF)
            ans = l;
    }
    return ans;
}
```